神经网络与深度学习

提纲

一、神经网络和深度学习

二、卷积神经网络

三、循环神经网络

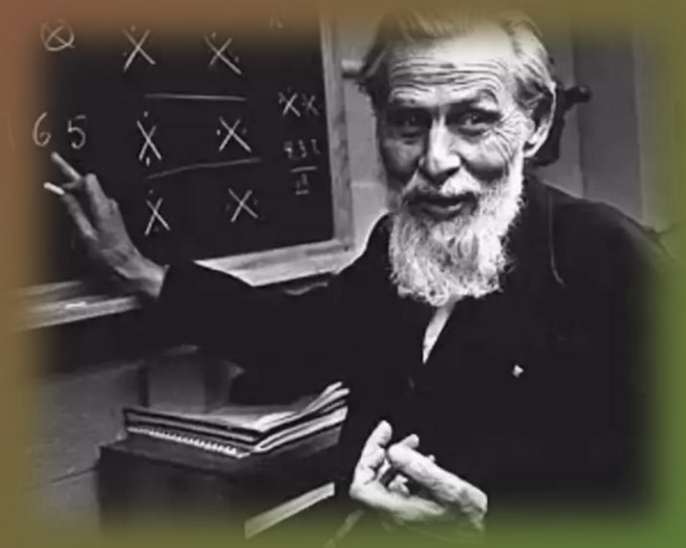
四、注意力机制与正则化

五、自然语言和计算机视觉应用

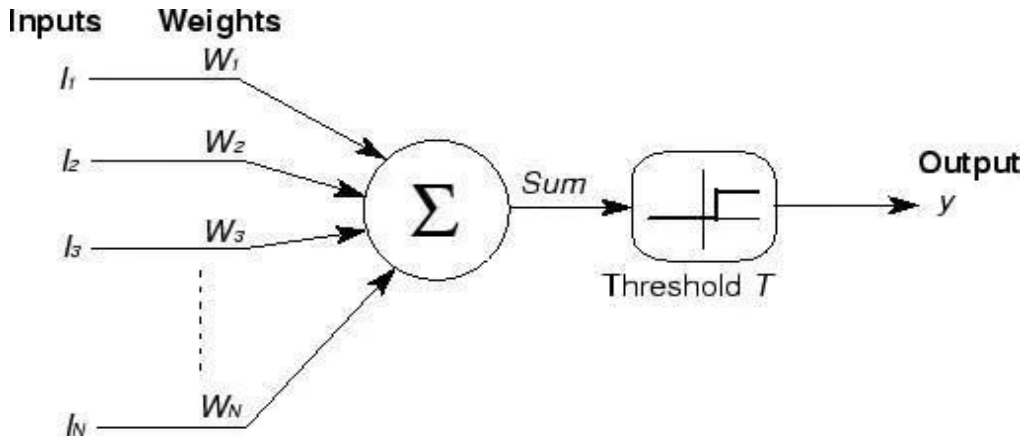
人工神经网络和深度学习---发展

1943 年， 神经科学家瓦伦· 麦卡洛克（Warren McCulloch） 和逻辑学家沃尔特· 皮茨（Walter Pitts）合作在《数学生物物理期刊》上发表了**神经网络的开山之作**： “A Logical Calculus of Ideas Immanent in Nervous Activity”, 提出了第一个神经元数学模型“**McCulloch–Pitts（MCP）neuron**”，是人类历史上第一次对大脑工作原理描述的尝试。

历史意义：我们在科学史上第一次知道了我们是怎么知道的（for the first time in the history of science, we know how we know)”



麦卡洛克（1898-1969）



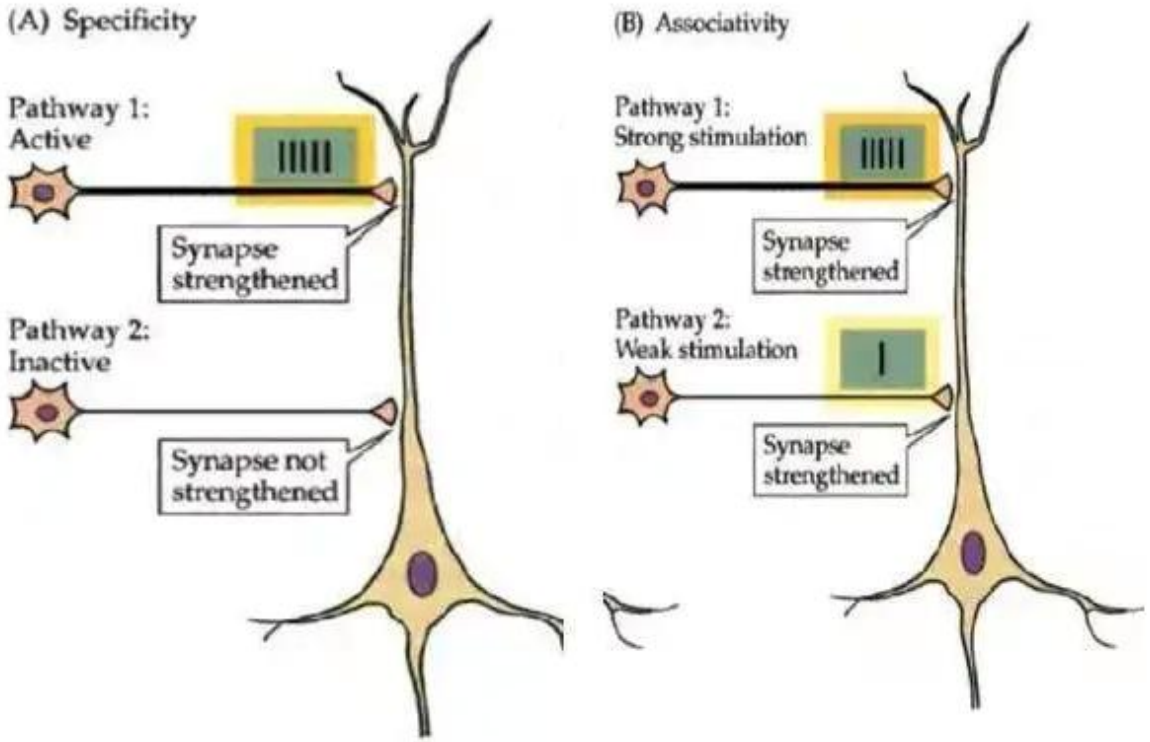
神经网络基本单元

MCP神经元

人工神经网络和深度学习---发展

1949 年，心理学家唐纳德·赫布（Donald Hebb）在《The Organization of Behavior》中对神经元之间连接强度的变化进行了分析，提出了赫布理论(Hebbian theory)：神经元之间持续**重复经验刺激**可导致突触传递效能增加(Neurons that fire together, wire together)”。其主要思想是根据两个**神经元的激发状态来调整连接关系**，以此实现对简单神经活动的模拟。

历史意义：神经元之间突触的强弱变化是学习与记忆的生理学基础

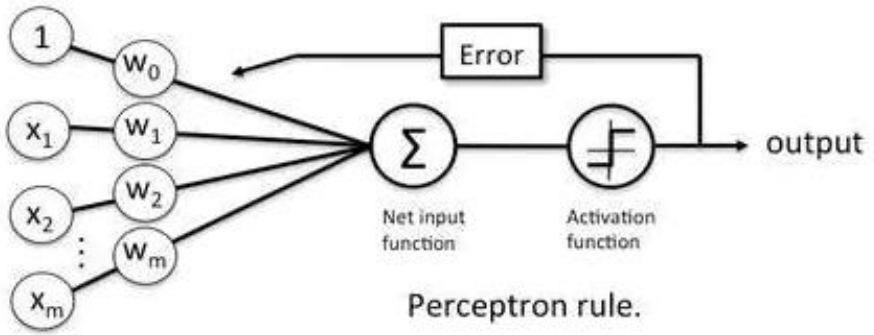


人工神经网络和深度学习---发展

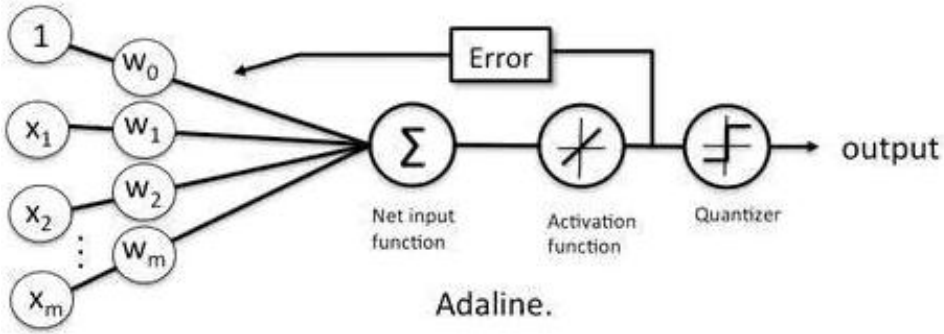
20世纪50年代就职于Cornell航空实验室的弗朗克·罗森布拉特（Frank Rosenblatt）提出“**感知机**（perceptron）”模型，其是一个仅包含输入层和输出层的两层神经网络，可完成两类问题的线性分类。**感知机是人工神经网络的第一个实际应用，标志着神经网络进入了新的发展阶段。**

1960年，斯坦福大学教授维德罗（Bernard Widrow）提出了Adaline可适应性算法。Adaline和感知机很相似，也是机器学习的鼻祖模型之一。Adaline网络和感知机的区别就是**将感知机的Step函数换为Linear线性函数。**

历史意义：感知机模型被誉为“Electronic 'Brain' Teaches Itself（能自学的电脑）”



Perceptron rule.



Adaline.

1969年，Marvin Minsky 和 Seymour Papert 从数学的角度证明了单层神经网络具有有限的功能，甚至在面对简单的“异或”逻辑问题时也显得无能为力。此后，**神经网络的研究陷入了很长一段时间的低迷期。**

人工神经网络和深度学习---发展

1974年，哈佛大学的一篇博士论文证明了在神经网络多加一层，并且利用“后向传播”（back-propagation）学习方法，**可以解决异或问题**。这篇论文的作者是沃波斯（Paul Werbos），他后来得到了IEEE神经网络学会的先驱奖。

神经网络在20世纪80年代的复兴归功于物理学家霍普菲尔德（John Hopfield）。1982年，那时在加州理工学院担任生物物理教授的霍普菲尔德，提出了一种新的神经网络，**可以解决一大类模式识别问题，还可以给出一类组合优化问题的近似解**。这种神经网络模型后来被称为霍普菲尔德网络。



霍普菲尔德（1933-）

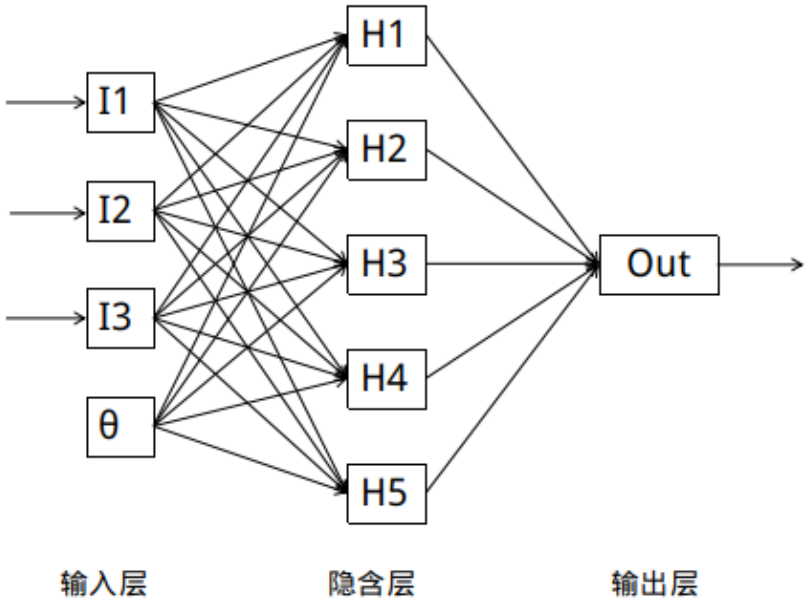
历史意义：霍普菲尔德模型的提出振奋了神经网络领域，使得神经网络的发展从低迷期转入繁荣期

人工神经网络和深度学习---发展

1986年神经网络之父辛顿（Hinton）**提出了适用于多层感知机MLP的BP算法**，并在层与层之间的传播过程中引入了 Sigmoid 函数，为神经网络引入了非线性，改良了单层感知机无法解决异或问题的缺陷。

BP算法包括前向传播的计算与反向传播的计算。如果是已经训练好的神经网络模型，仅使用前向传播即可预测出结果。神经网络的训练还需要反向传播的参与。当前向传播预测值与真实值有出入的时候，利用**梯度下降**的方法，在反向传播中使模型权重改变，让模型权重更加贴合真实值。常见的梯度更新优化器有如下四种，它们分别是：SGD、Momentum、AdaGrad、Adam。

历史意义：克服了第一代神经网络的弱点，为多层神经网络的学习训练与实现提供了一种切实可行的解决途径。



BP神经网络

人工神经网络和深度学习---发展

2006年，辛顿在Science等期刊上发表题为《Reducing the Dimensionality of Data with Neural Networks》的文章，并**首次提出“深度信念网络（deep belief network）”模型**（在相关分类任务中可取得性能超过传统浅层学习模型（BP神经网络）的结果。

这篇文章是一个分水岭，拉开了深度学习大幕，标志着深度学习的诞生。从此，历史这样写就：从感知机提出，到BP算法应用以及2006年以前的历史被称为浅层学习，以后的历史被称为深度学习。



辛顿（1947-）

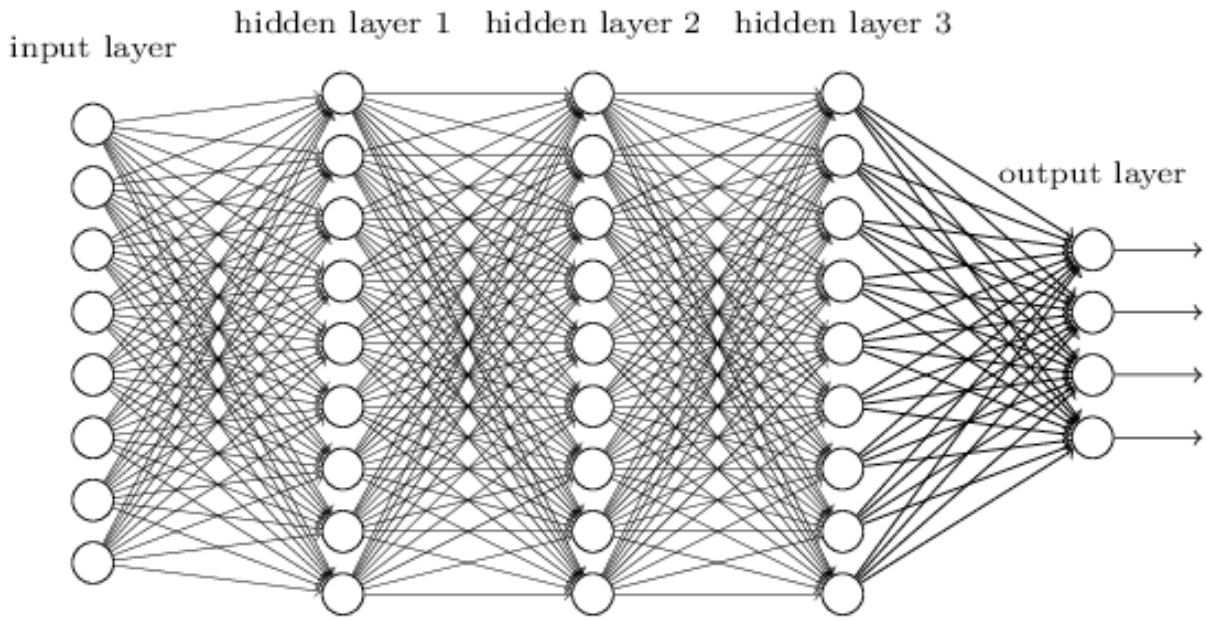
历史意义：有别于分段式学习，端到端(end to end)的深度学习框架建立

人工神经网络和深度学习---发展

第三代神经网络代表：

- RNN、LSTM
- CNN
- Word2vec
- BERT、Transformer、LLM

历史意义：从造人和造脑到更通用人工智能



深度神经网络（DNN）

引发AI第三次崛起

语言大模型

人工神经网络和深度学习---前馈神经网络

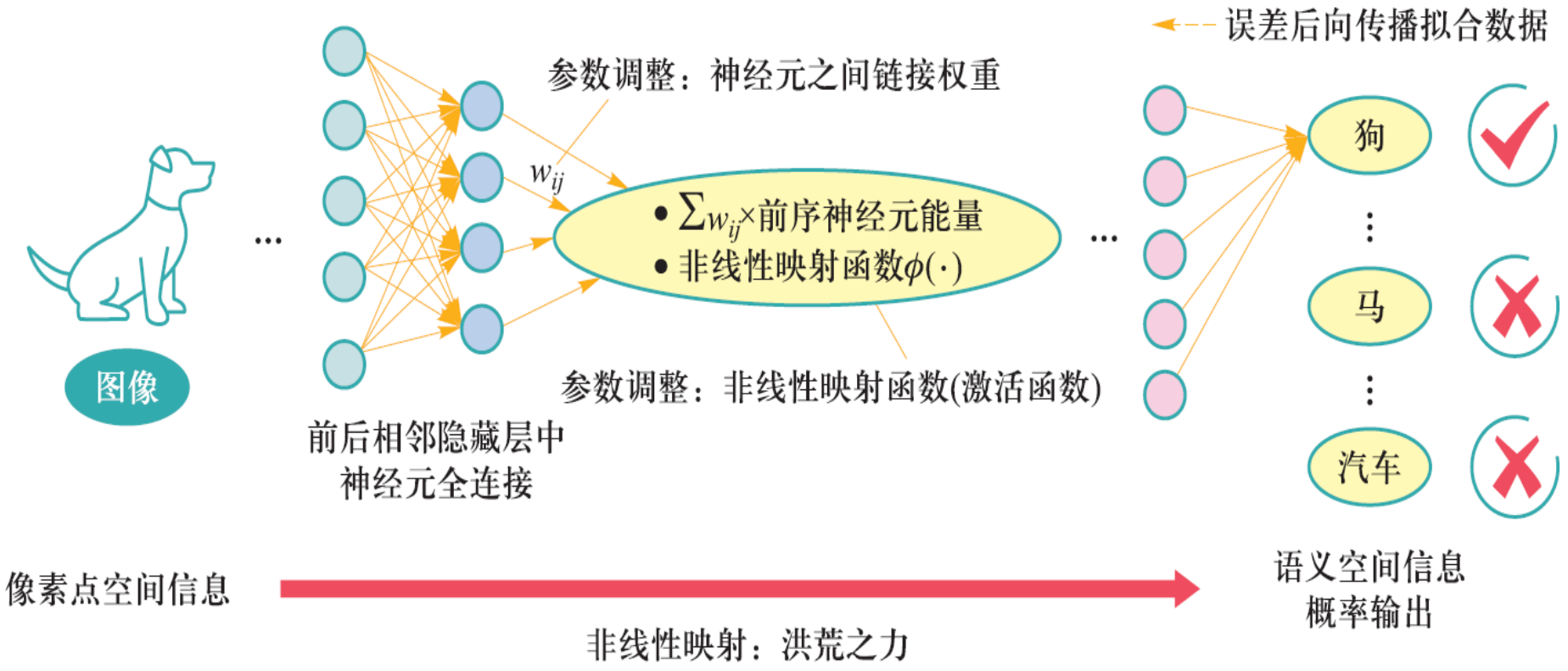
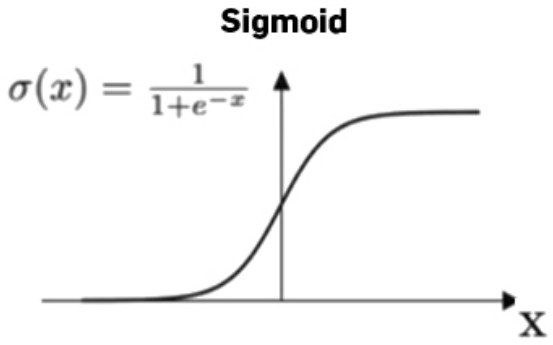
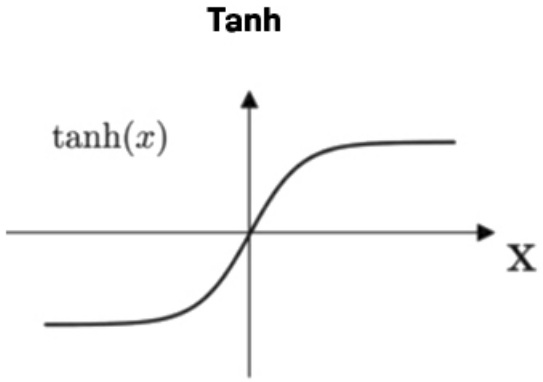
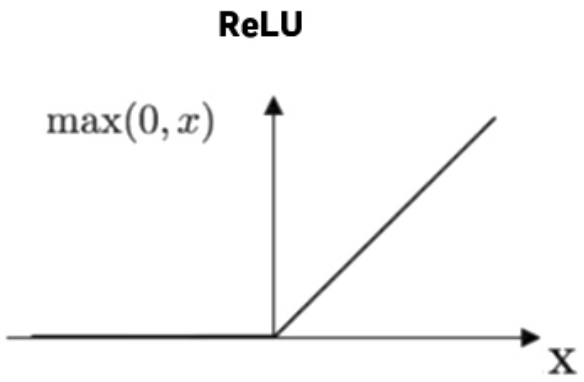


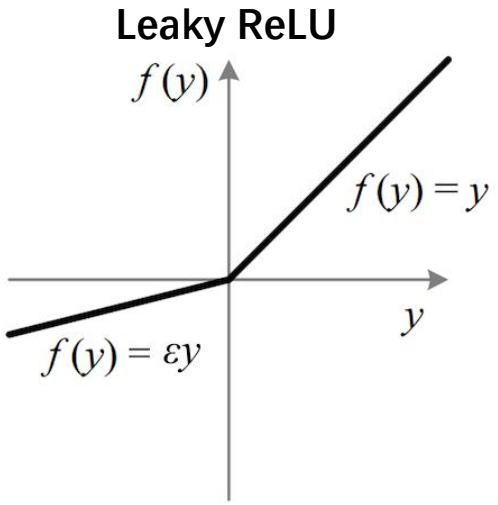
图 5.4　前馈神经网络模型

人工神经网络和深度学习---常见激活函数









人工神经网络和深度学习---常见激活函数

sigmoid函数

$$f(x)=\frac{1}{1+e^{-x}}$$

优点：1）概率形式输出；2）单调递增。3）非线性变化。

缺点：$f'(x)=(\frac{1}{1+e^{-x}})'=\frac{e^{-x}}{(1+e^{-x})^2}=\frac{1+e^{-x}-1}{(1+e^{-x})^2}=\frac{1}{(1+e^{-x})}\left(1-\frac{1}{(1+e^{-x})}\right)=f(x)(1-f(x))$

1）反向传播时，易出现梯度接近于0的情况，即梯度消失（vanishing gradient）的问题

2）函数指数形式，计算复杂度高

人工神经网络和深度学习---常见激活函数

ReLU函数

$$f(x)=\begin{cases}0, for\ x<0\\ x, for\ x\geq 0\end{cases}$$ 或者$f(x)=\max(0,x)$

ReLU函数的导数为：$f'(x)=\begin{cases}0, for\ x<0\\ 1, for\ x\geq 0\end{cases}$

优点：1）当输入$x\geq 0$时，ReLU的导数为常数，不会出现梯度消失、梯度饱和问题。

2）训练神经网络能加速收敛，计算简单，复杂度低，不需要进行指数运算。

3）当$x<0$时， ReLU的梯度总是0，神经元“死亡”， 参与分类等任务神经元数目稀缺，这种稀疏性可以在一定程度上克服过拟合现象。

缺点：极端情况会出现大量 “死亡”神经元。

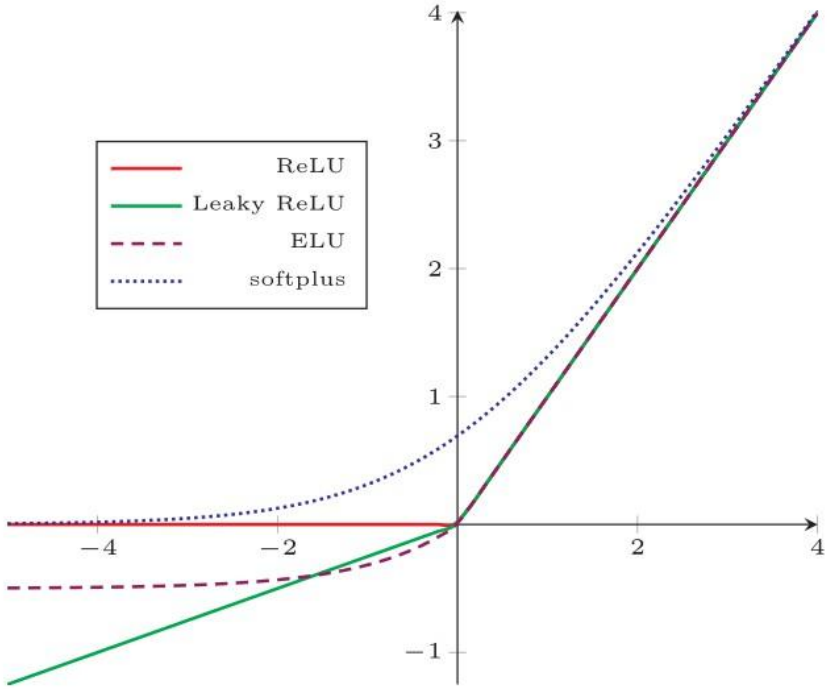
使用其他激活函数解决这个问题，如 Leaky ReLU 或Parametric ReLU，它们在x < 0时梯度取值非零。

ReLU函数---隐藏层常用的默认选择

人工神经网络和深度学习---常见激活函数

$$\mathrm{ReLU}(x) = \begin{cases} x & x \geq 0 \\ 0 & x < 0 \end{cases} = \max(0, x).$$

$$\mathrm{LeakyReLU}(x) = \begin{cases} x & \text{if } x > 0 \\ \gamma x & \text{if } x \leq 0 \end{cases} = \max(0, x) + \gamma \min(0, x)$$



当前的大模型中用的什么激活函数?

$$\mathrm{PReLU}_i(x) = \begin{cases} x & \text{if } x > 0 \\ \gamma_i x & \text{if } x \leq 0 \end{cases} = \max(0, x) + \gamma_i \min(0, x)$$

$$\mathrm{ELU}(x) = \begin{cases} x & \text{if } x > 0 \\ \gamma(\exp(x) - 1) & \text{if } x \leq 0 \end{cases} = \max(0, x) + \min(0, \gamma(\exp(x) - 1))$$

$$\mathrm{softplus}(x) = \log(1 + \exp(x))$$

人工神经网络和深度学习---常见激活函数

Softmax激活函数

$$y_i = \text{softmax}(x_i) = \frac{e^{x_i}}{\sum_{j=1}^{k} e^{x_j}}$$

Softmax函数一般用于多分类任务最后一层

Softmax函数将输入向量x_i映射到第i个类别的概率$y_i, 0 < y_i < 1$。

图5.5给出了三类分类问题的Softmax输出示意图。。

由于Softmax输出结果的值累加起来为1，因此可将输出概率最大的作为分类目标（图5.5中被分类为第一类）。

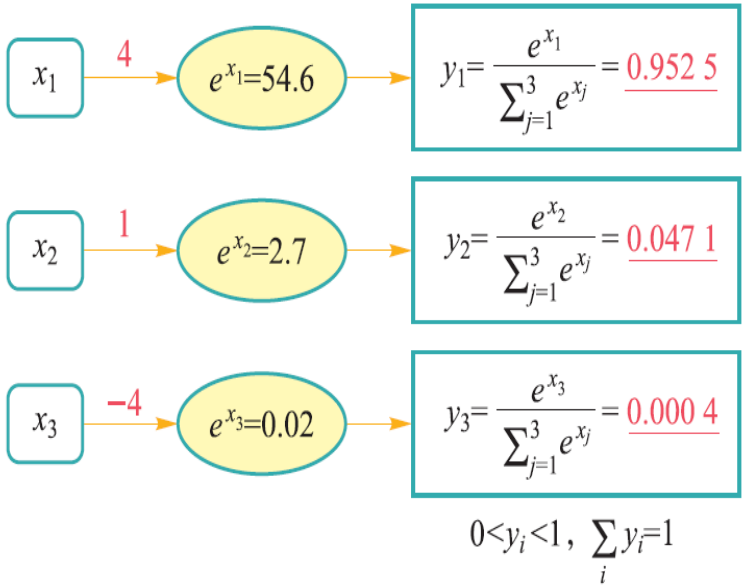
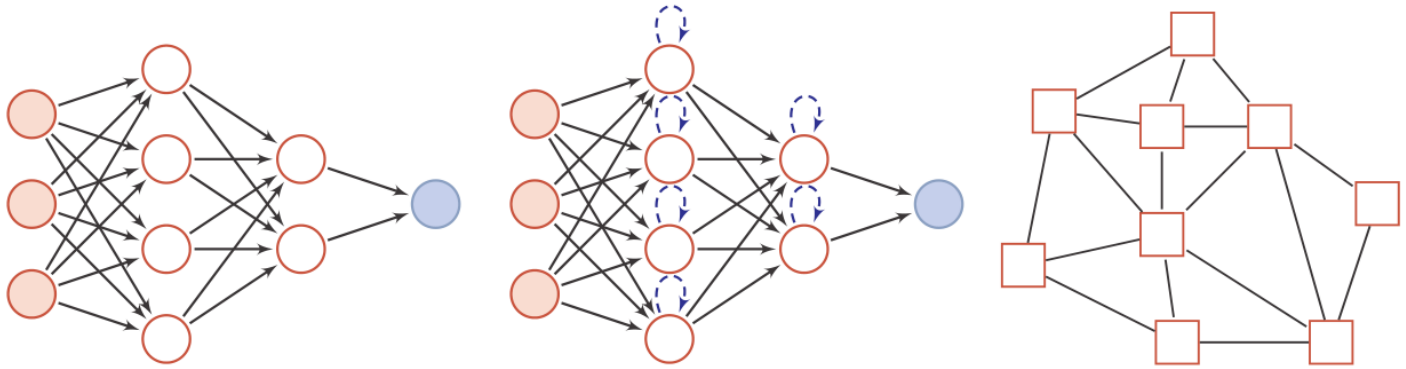


图 5.5　三分类问题的 softmax 输出示意

人工神经网络和深度学习---网络结构

- 人工神经网络由神经元模型构成，这种由许多神经元组成的信息处理网络具有并行分布结构。



(a) 前馈网络　(b) 记忆网络　(c) 图网络

圆形节点表示一个神经元，方形节点表示一组神经元。

人工神经网络和深度学习---神经网络参数优化

- 多层神经网络结构设计完成后，需要对神经网络中参数进行优化学习，以使得神经网络能够将给定输入数据映射到所期望的输出语义空间，完成分类识别等任务。

- 神经网络参数优化是一个监督学习的过程，通过损失函数来计算模型预测值$\hat{y}_i$与真实值y_i之间误差或损失，利用其来优化模型参数，以便让神经网络按照x_i的真实信息y_i输出。

- 模型利用反向传播算法将损失函数计算所得误差从输出端出发、由后向前传递给神经网络中每个单元，然后通过梯度下降算法对神经网络中参数进行更新。随着迭代的进行，模型预测所得$\hat{y}_i$与真实值y_i之间的误差逐步缩小，模型预测准确率得以提升，预测的能力得以增强，当迭代达到一定轮次或准确率达到一定水平时，则可认为模型参数已被优化完毕。

人工神经网络和深度学习---损失函数

- 损失函数（loss function）又称为代价函数（cost function），用来计算模型预测值与真实值之间的误差。损失函数是神经网络设计中的一个重要组成部分。通过定义与任务相关的良好损失函数，在训练过程中可根据损失函数来计算神经网络的误差大小，进而优化神经网络参数。
- 两种最常用的损失函数：均方误差损失函数和交叉熵损失函数。

均方误差损失函数

$$\mathrm{MSE} = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

交叉熵损失函数

$$\mathrm{H}(y_i, \hat{y}_i) = -y_i * log\hat{y}_i$$

人工神经网络和深度学习---损失函数

交叉熵损失函数 $\mathrm{H}(y_i, \hat{y}_i) = -y_i * log\hat{y}_i$

- “熵”被用来表示热力学系统所呈现的无序程度。香农将这一概念引入信息论领域，提出了“信息熵”概念，通过对数函数来测量信息的不确定性。
- 交叉熵（cross entropy）是信息论中的重要概念，主要用来度量两个概率分布间的差异。y_i是样本x_i分类的真实概率、$\hat{y}_i$是模型预测概率。
- 交叉熵刻画了两个概率分布之间的距离，旨在描绘通过概率分布 $\hat{y}_i$ 来表达概率分布 y_i 的困难程度，两个概率分布y_i和$\hat{y}_i$越接近，交叉熵越小。即，预测类别分布概率与实际类别分布概率之间的差距越小，交叉熵就越小。

人工神经网络和深度学习---损失函数

这里以一个示例子来说明这个事实。假设模型 1 和模型 2 分别对三个分别属于类别 1、类别 2 和类别 3 的样本进行分类，得到如表 5.2 所示的分类结果。

表 5.2　两个模型的分类结果

样本	真实值	模型 1 预测结果	模型 2 预测结果
样本 1（类别 1）	0 0 1	0.3 0.3 0.4	0.1 0.2 0.7
样本 2（类别 2）	0 1 0	0.3 0.4 0.3	0.1 0.7 0.2
样本 3（类别 3）	1 0 0	0.1 0.2 0.7	0.3 0.4 0.3

模型 1 在三个样本上的交叉熵损失平均值为

$$\frac{-(0\times\log 0.3+0\times\log 0.3+1\times\log 0.4+0\times\log 0.3+1\times\log 0.4+0\times\log 0.3+1\times\log 0.1+0\times\log 0.2+0\times\log 0.7)}{3}$$

$$=1.37$$

模型 2 在三个样本上的交叉熵损失平均值为

$$\frac{-(0\times\log 0.1+0\times\log 0.2+1\times\log 0.7+0\times\log 0.1+1\times\log 0.7+0\times\log 0.2+1\times\log 0.3+0\times\log 0.4+0\times\log 0.3)}{3}$$

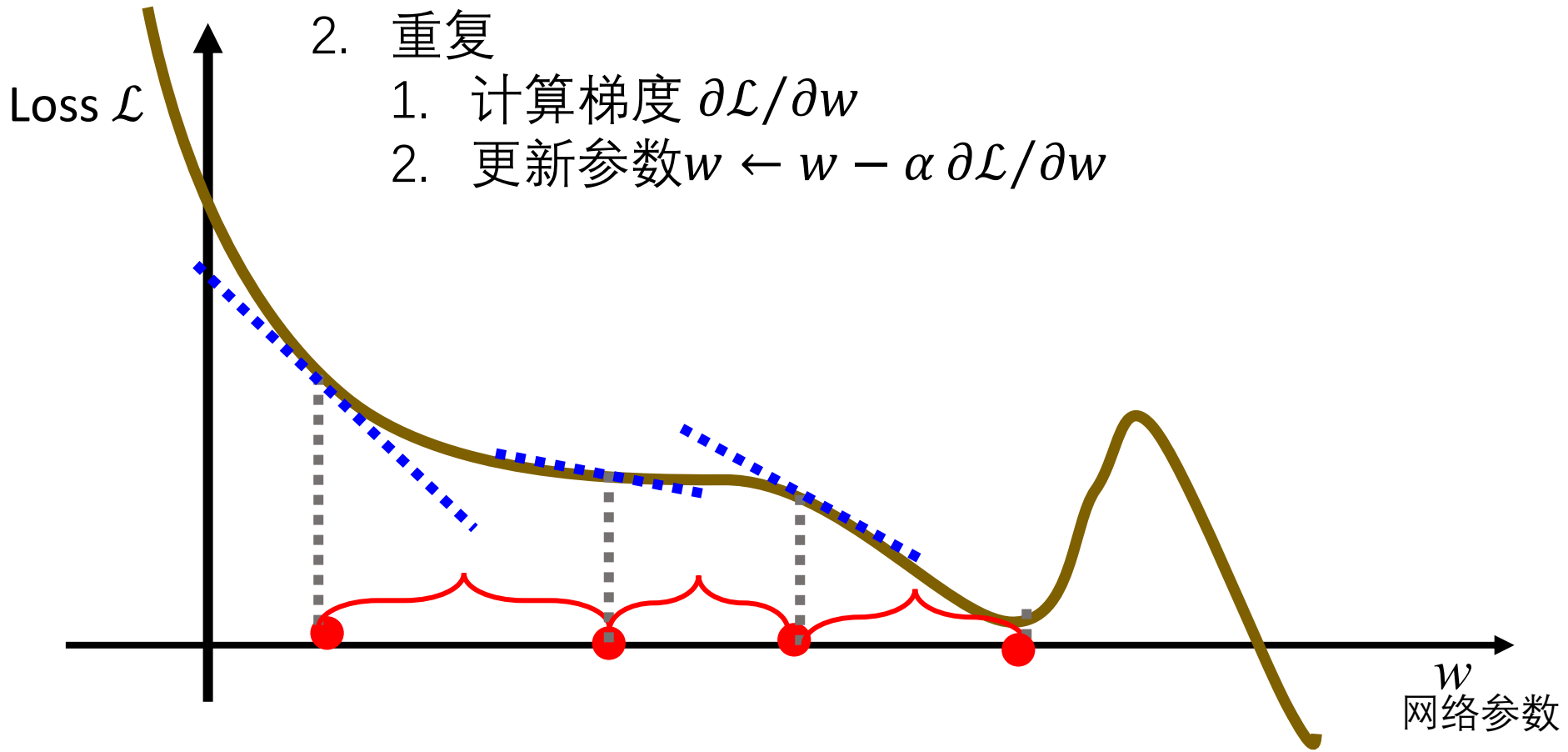
$$=0.63$$

可见，交叉熵损失函数准确给出了模型 2 预测性能优于模型 1 预测性能的事实。如果模型预测值与真实值一致，则交叉熵损失为零。

人工神经网络和深度学习---梯度下降

1. 初始化w
2. 重复
   1. 计算梯度 $\partial\mathcal{L}/\partial w$
   2. 更新参数$w \leftarrow w - \alpha\,\partial\mathcal{L}/\partial w$

梯度: $\frac{\partial f(w)}{\partial w} = \lim_{\Delta w \to 0} \frac{f(w+\Delta w)}{\Delta w}$



人工神经网络和深度学习---梯度下降

权值V_{ti}的修正公式推导

由$\Delta V_{ti} \approx \frac{\partial V_{ti}}{\partial T}$，**要满足 1)** 总误差$E_k$随时间$T$递减，即$\frac{\partial E_k}{\partial T} \le 0$，且

2) E_k沿V_{ti}最快递减（递减变化率最大），即沿梯度$\frac{\partial E_k}{\partial V_{ti}}$变化

由于 $\frac{\partial E_k}{\partial T} = \sum_{t=1}^{q} \frac{\partial E_k}{\partial V_{ti}} \frac{\partial V_{ti}}{\partial T}$，当$\frac{\partial V_{ti}}{\partial T} = -\frac{\partial E_k}{\partial V_{ti}}$时，有$\frac{\partial E_k}{\partial T} = -\sum_{t=1}^{q} (\frac{\partial E_k}{\partial V_{ti}})^2 \le 0$

设 $\Delta V_{ti} \approx \frac{\partial V_{ti}}{\partial T} = -\alpha \frac{\partial E_k}{\partial V_{ti}} = -\alpha \frac{\partial E_k}{\partial O_t} \times \frac{\partial O_t}{\partial net_t} \times \frac{\partial net_t}{\partial V_{ti}}$，$\alpha$是常数

这里 $net_t = \sum_{i=1}^{n} V_{ti} y_i - r_t$

人工神经网络和深度学习---梯度下降

1） $\frac{\partial net_t}{\partial V_{ti}} = y_i$

2） $\frac{\partial E_k}{\partial O_t} = \frac{\partial}{\partial O_t}(\sum_{t=1}^{q}(D_t^{(k)} - O_t^{(k)})^2 / 2) = -(D_t^{(k)} - O_t^{(k)})$

3） $\frac{\partial O_t}{\partial net_t} = \frac{\partial}{\partial net_t}(f(net)) = f(net_t)[1 - f(net_t)] = O_t(1 - O_t)$

这里 $O_t = f(\sum_{i=1}^{n} V_{ti} y_i - r_t) = f(net_t)$

$$f(x) = \frac{1}{1+e^{-x}} \qquad f'(x) = f(x)[1 - f(x)]$$

综合有

$$\frac{\partial V_{ti}}{\partial T} = -\alpha \frac{\partial E_k}{\partial V_{ti}} = -\alpha \frac{\partial E_k}{\partial O_t} \times \frac{\partial O_t}{\partial net_t} \times \frac{\partial net_t}{\partial V_{ti}}$$

$$= -\alpha(-(D_t^{(k)} - O_t^{(k)}))O_t(1 - O_t)y_i = \alpha\delta^{(k)}O_t(1 - O_t)y_i = \alpha h_t^{(k)} y_i$$

这里 $h_t^{(k)} = \delta^{(k)}O_t(1 - O_t) = \frac{\partial E_k}{\partial net_t}$

隐层与输出层连接权的修正：

V_{ti} (K+1) $= V_{ti}$ (K) $+ \Delta V_{ti}$

$\Delta V_{ti} = \alpha_1 \delta_t^{(k)} O_t(1-O_t) Y_i = \alpha h_t^{(k)} Y_i$,

$\delta_t^{(k)} = D_t^{(k)} - O_t^{(k)}$, Y_i 是隐节点输出

人工神经网络和深度学习---梯度下降

权值 w_{ij} 的修正公式推导

同理，也要求误差函数随时间变化递减，并沿 w_{ij} 变化率最大的方向递减，此时隐层节点误差依赖于输出层误差的传递。可设

$$\Delta W_{ij} \approx \frac{\partial W_{ij}}{\partial t} = -\beta \frac{\partial E_k}{\partial W_{ij}} = -\beta \frac{\partial E_k}{\partial y_i} \times \frac{\partial y_i}{\partial net_i} \times \frac{\partial net_i}{\partial W_{ij}}, \quad \text{其中 } net_i = \sum_{i=1}^{n} W_{ij} x_j - \theta_i$$

1) $$\frac{\partial net_i}{\partial W_{ij}} = x_j$$

2) $$\frac{\partial y_i}{\partial net_i} = \frac{\partial}{\partial net_i}(f(net)) = f(net_i)[1 - f(net_i)] = y_i(1 - y_i)$$

3) $$\frac{\partial E_k}{\partial y_i} = \sum_{l=1}^{q} \frac{\partial E_k}{\partial net_l} \frac{\partial net_l}{\partial y_i} = \sum_{l=1}^{q} -(D_l^{(k)} - O_l^{(k)}) O_l^{(k)} (1 - O_l^{(k)}) V_{li} = \sum_{l=1}^{q} -h_l^{(k)} V_{li}$$

这里，以前面 $\delta^{(k)} O_t (1 - O_t) = \frac{\partial E_k}{\partial net_t}$，且 $\frac{\partial net_l}{\partial y_i} = V_{li}$ 代入得到. 综合有

$$\Delta W_{ij} \approx \frac{\partial W_{ij}}{\partial t} = -\beta \frac{\partial E_k}{\partial W_{ij}} = -\beta \frac{\partial E_k}{\partial y_i} \times \frac{\partial y_i}{\partial net_i} \times \frac{\partial net_i}{\partial W_{ij}}$$

$$= -\beta \sum_{l=1}^{q} (-h_l^{(k)} V_{li}) y_i (1 - y_i) x_j$$

$$= \beta e_i^{(k)} x_j$$

输入层与隐层连接权值修正公式

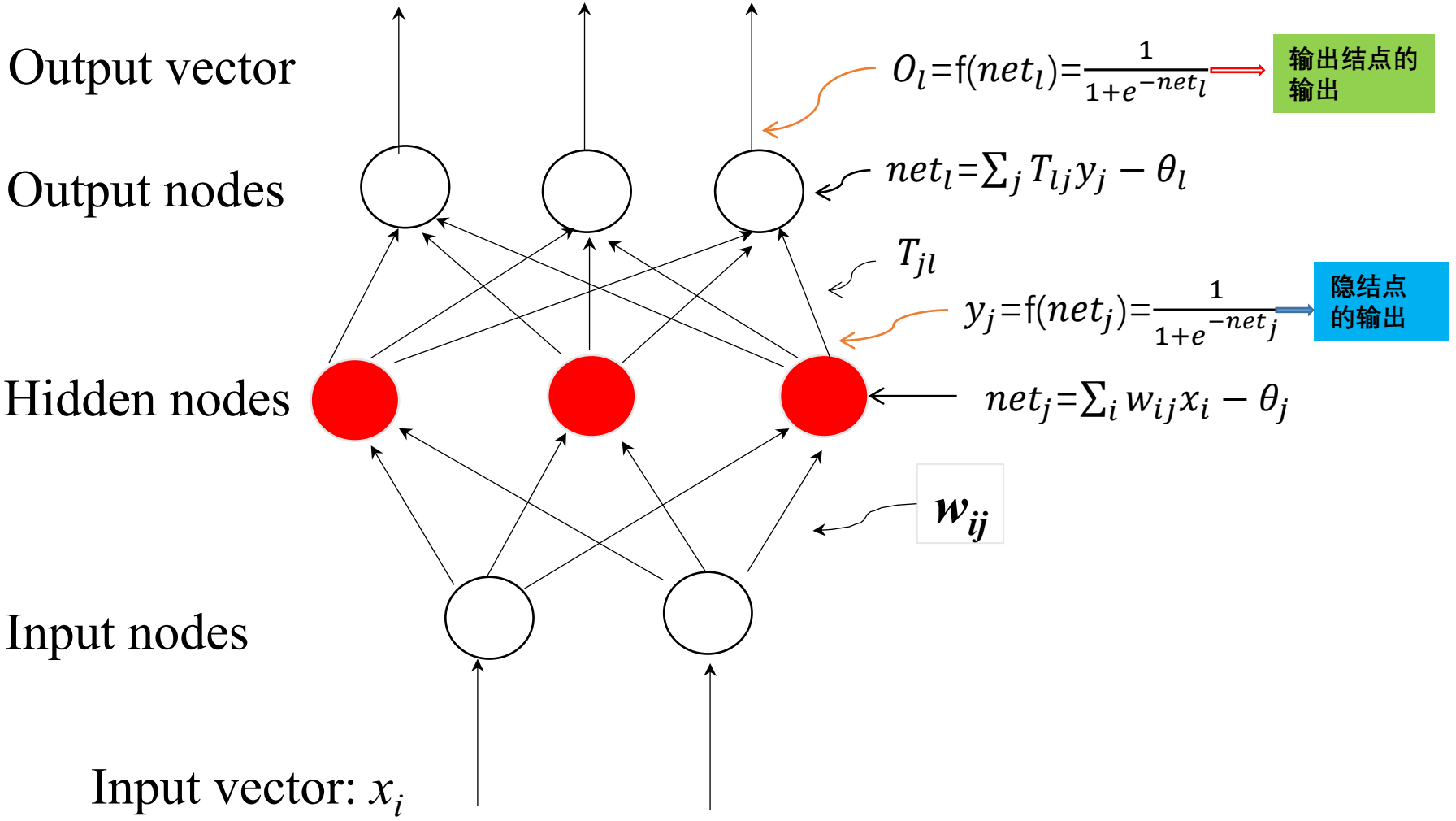
$$w_{ij}(K+1) = w_{ij}(K) + \Delta w_{ij},$$

$$\Delta w_{ij} = \beta_1 \sum_{t=1}^{q} h_t^{(k)} V_{ti} Y_i (1 - Y_i) x_j,$$

$h_t^{(k)} = \delta_t^{(k)} O_t^{(k)} (1 - O_t^{(k)})$，$x_j$ 是输入节点输入。

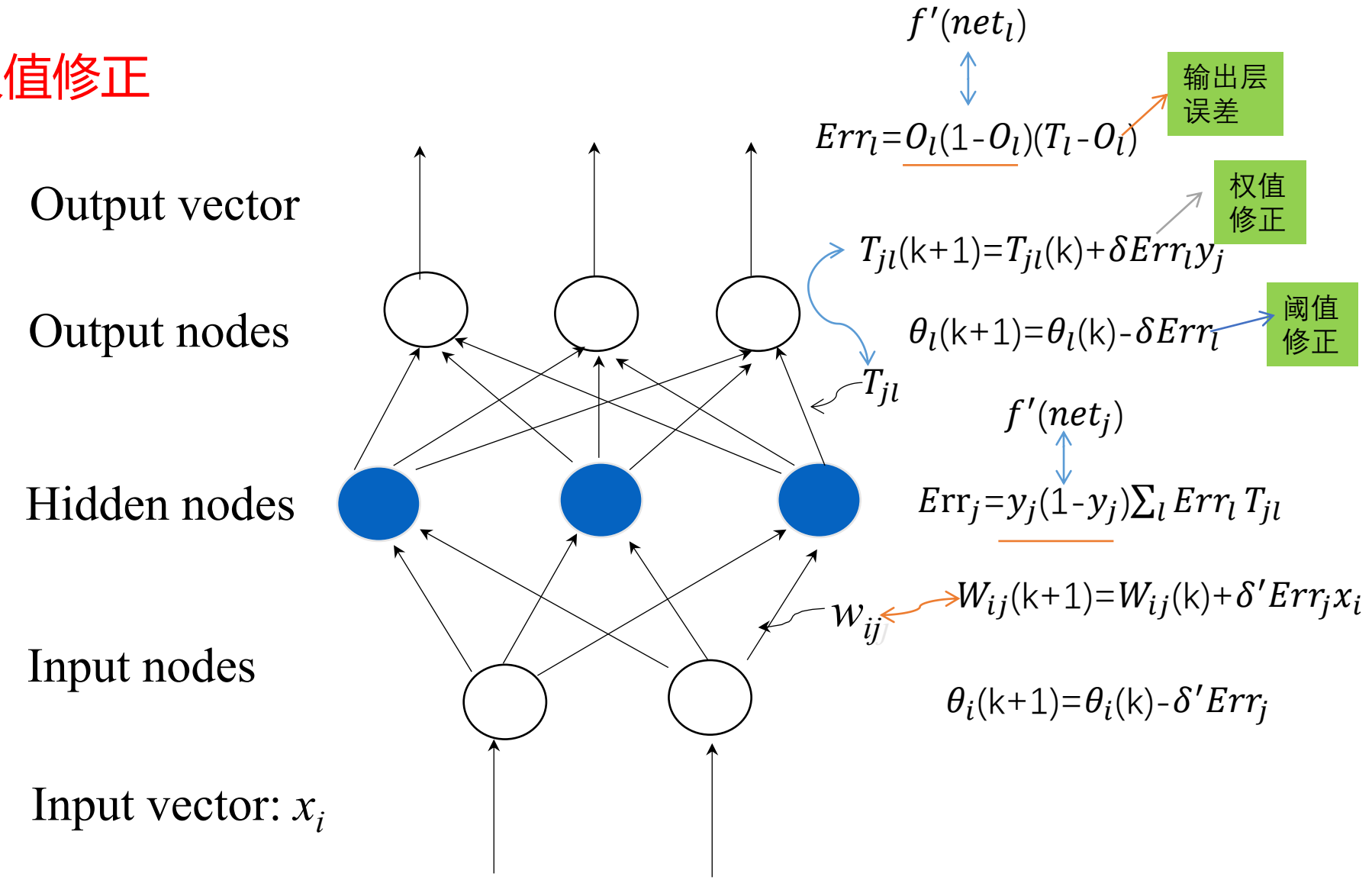
人工神经网络和深度学习---传播

- 信号正向传播



人工神经网络和深度学习---传播

- 反向传播权值修正



卷积神经网络---概论

- 卷积神经网络（Convolutional Neural Networks，CNN）
- 一种前馈神经网络
- 受生物学上感受野（Receptive Field）的机制而提出的
  - 在视觉神经系统中，一个神经元的感受野是指视网膜上的特定区域，只有这个区域内的刺激才能够激活该神经元。

- 卷积神经网络有三个结构上的特性：
  - 局部连接
  - 权重共享
  - 空间或时间上的次采样

卷积神经网络---概论

一个例子

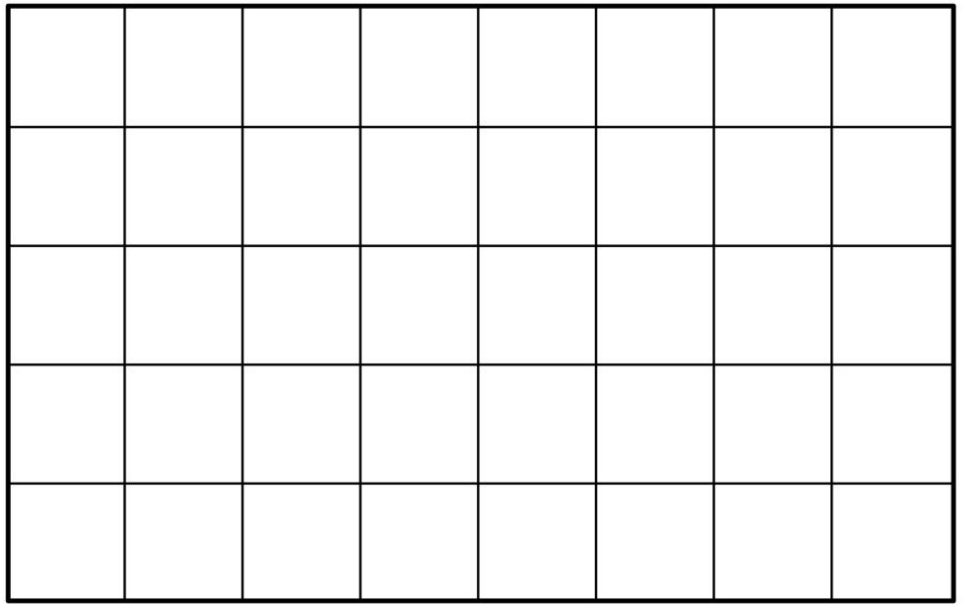




卷积神经网络---概论

一个例子



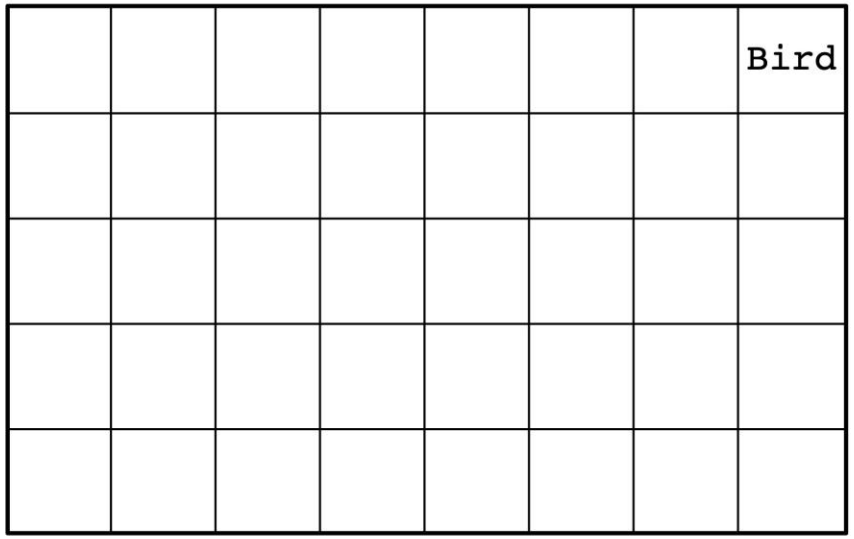




卷积神经网络---概论

一个例子







卷积神经网络---概论

一个例子

- 简单且理想的情况
  - 图片中的每个物体都恰好包在一个块中
  - 物体大小和块的大小接近

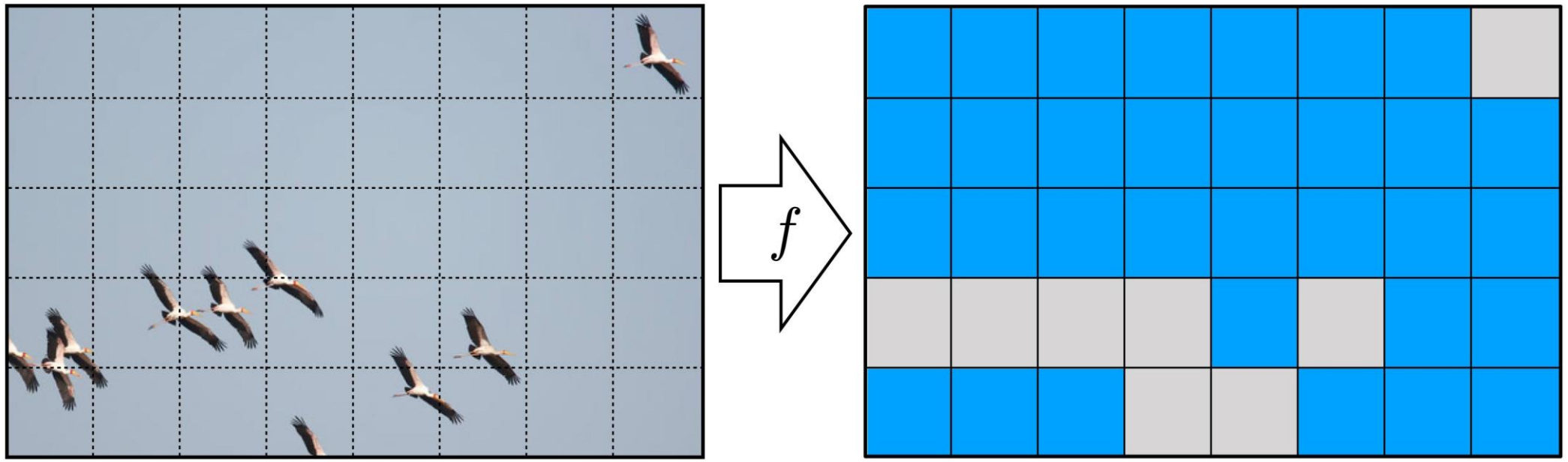


Sky	Sky	Sky	Sky	Sky	Sky	Sky	Bird
Sky	Sky	Sky	Sky	Sky	Sky	Sky	Sky
Sky	Sky	Sky	Sky	Sky	Sky	Sky	Sky
Bird	Bird	Bird	Sky	Bird	Sky	Sky	Sky
Sky	Sky	Sky	Bird	Sky	Sky	Sky	Sky

卷积神经网络---概论

问题

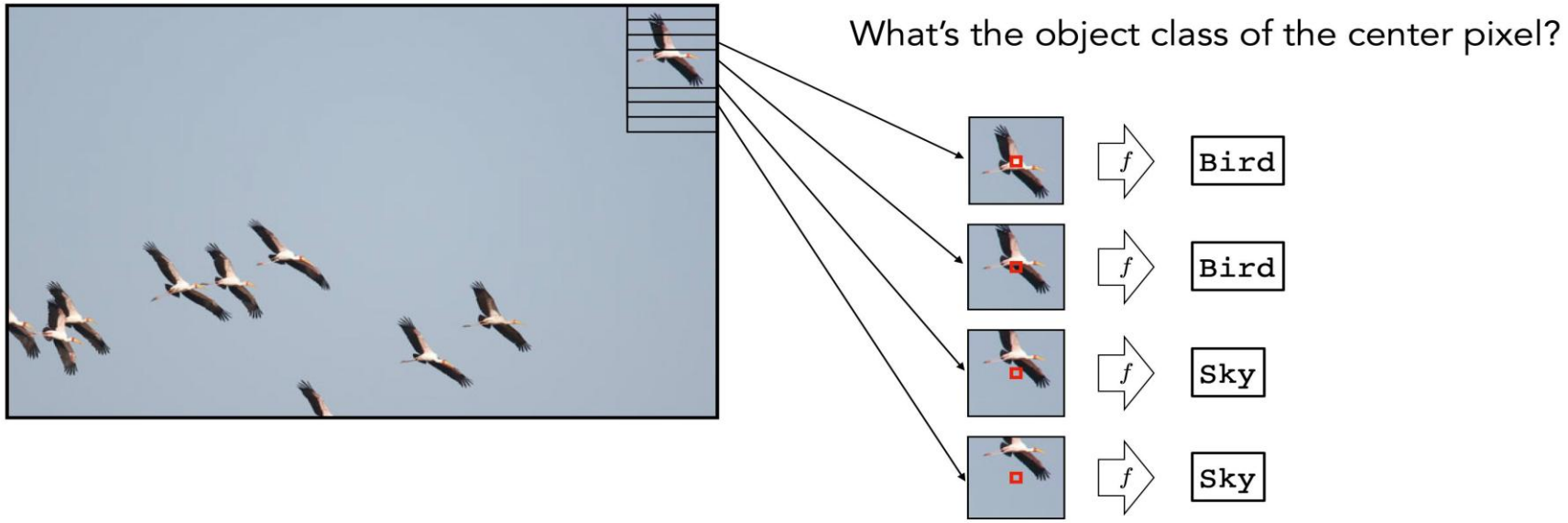
- 如果图片中的物体比块大得多该怎么办?
- 如果一个物体处于多个块中怎么办?



卷积神经网络---概论

如何更加有效地提取局部特征?

- 改进小块的划分方式:
  - 以每个像素为中心划分出一系列相互重叠的稠密块
  - 相邻块之间的偏移均为一个像素
  - 将每个块的预测结果作为这个块中心像素点的预测结果

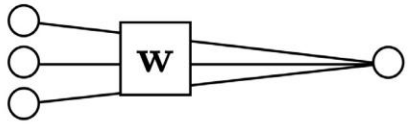


卷积神经网络---卷积

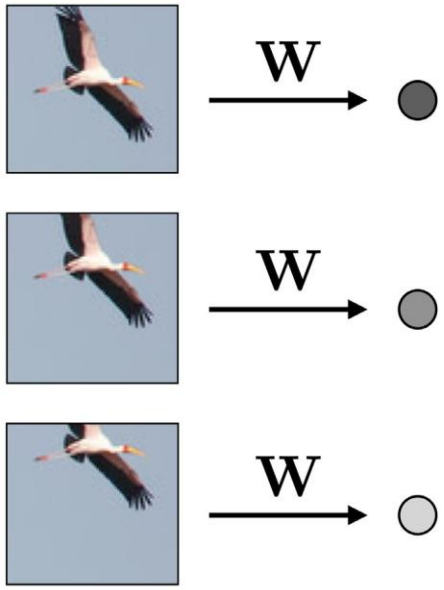
卷积

- 如何处理每个小块?
  - 卷积核: 权重矩阵W
  - 卷积操作：加权求和
- 权重共享

W computes a weighted sum of all pixels in the patch



- 为何叫做“卷积”？
  - 无需翻转，本质是“互相关”
  - 早期受信号处理领域启发

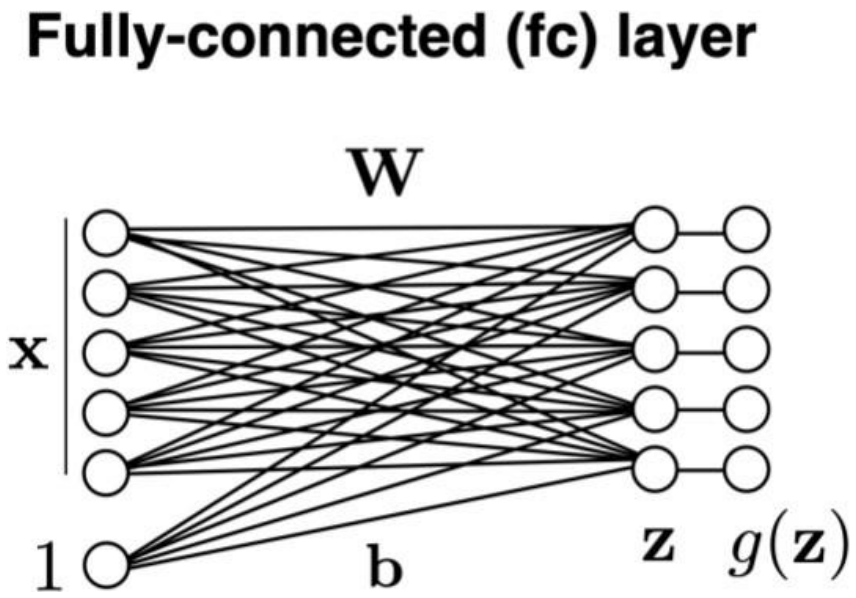


W is a **convolutional kernel** applied to the full image!

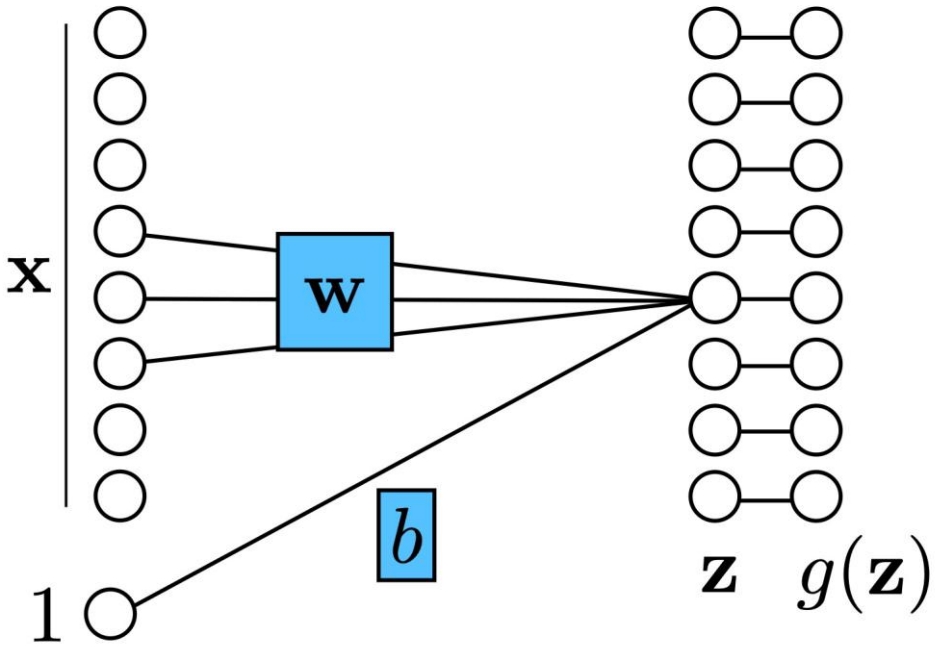
卷积神经网络---卷积

全连接层 vs. 局部连接层

全连接：相关每个神经元都与前一层所有神经元相关。

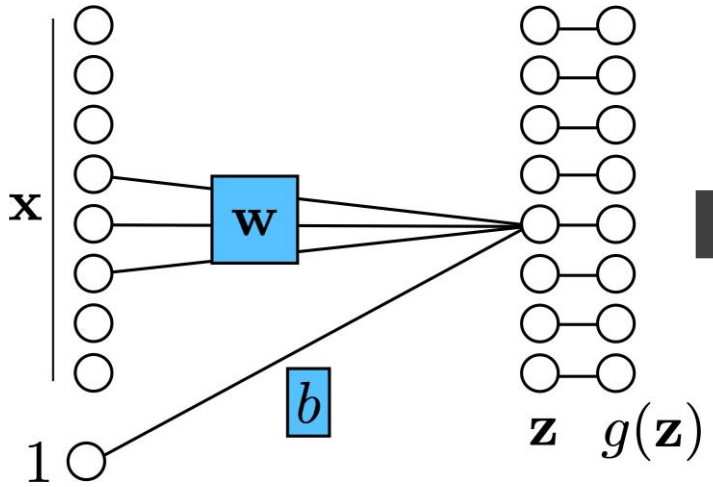


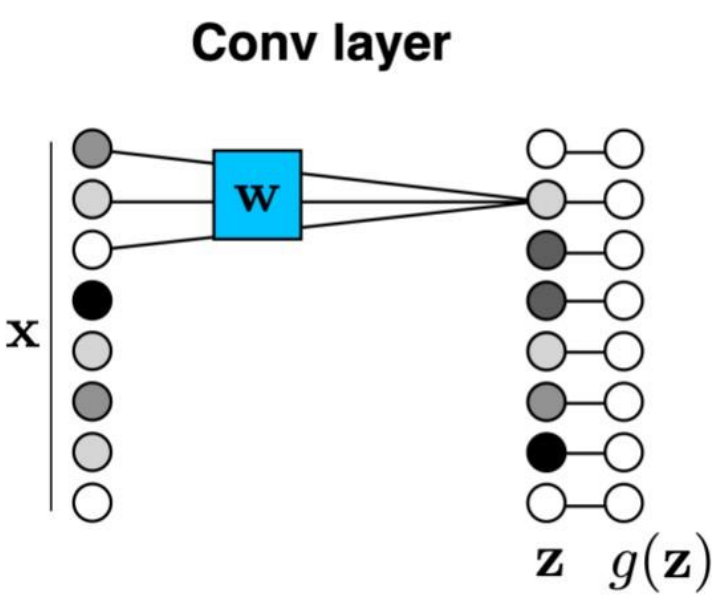
局部连接：输出仅和个别输入分量相关权重/参数共享：局部参数w是共享的



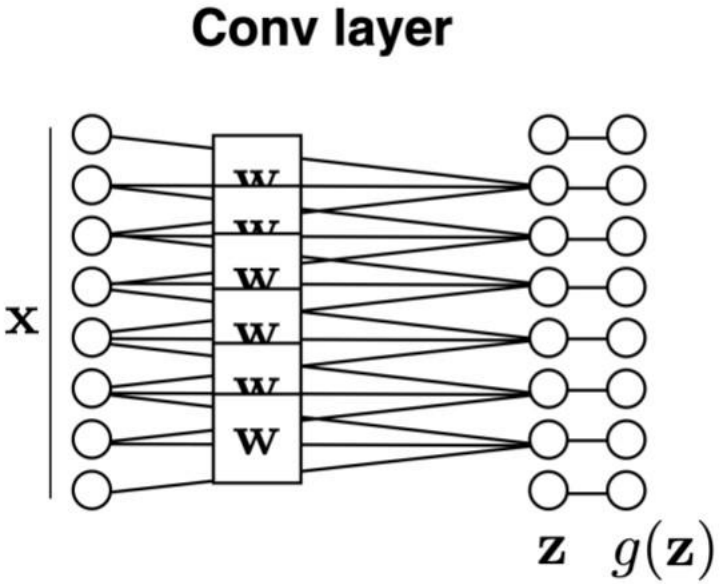
卷积神经网络---卷积

从局部连接到卷积层





$$\mathbf{z} = \mathbf{w} \circ \mathbf{x} + \mathbf{b}$$



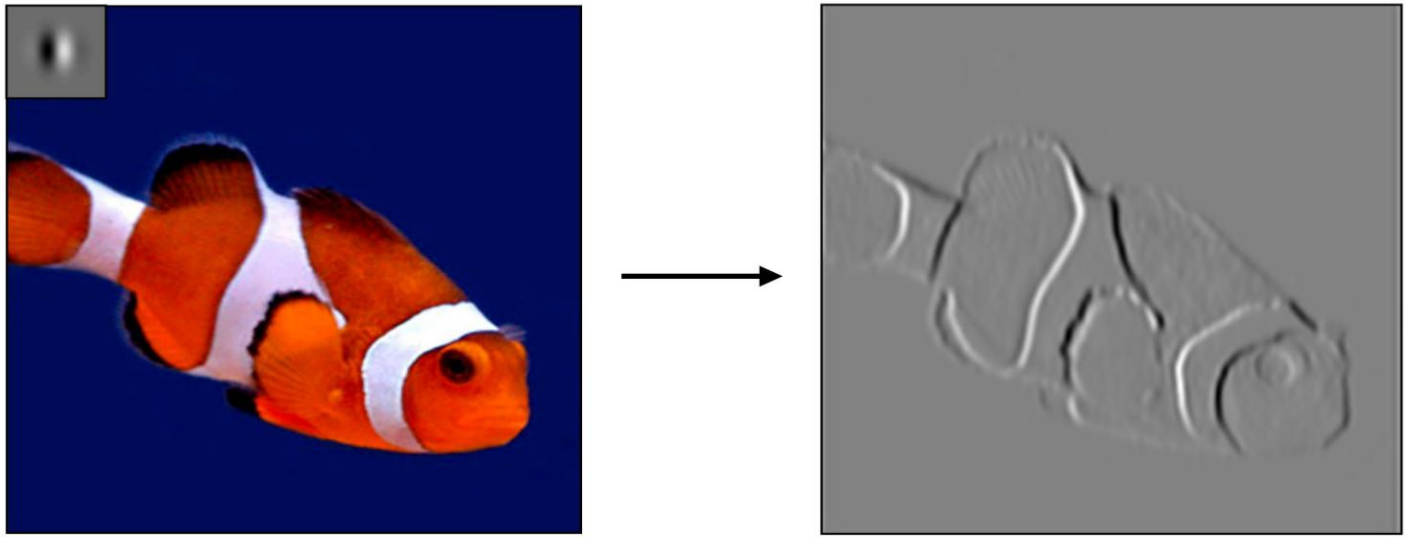
$$\mathbf{z} = \mathbf{w} \circ \mathbf{x} + \mathbf{b}$$

参数共享
(Weight Sharing)

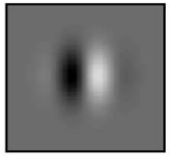
卷积神经网络---卷积

卷积核：Conv2D

- 线性、平移不变的计算操作 *（算子，Operator），也成为滤波器 filter*



示例：一个提取边缘信息的卷积核

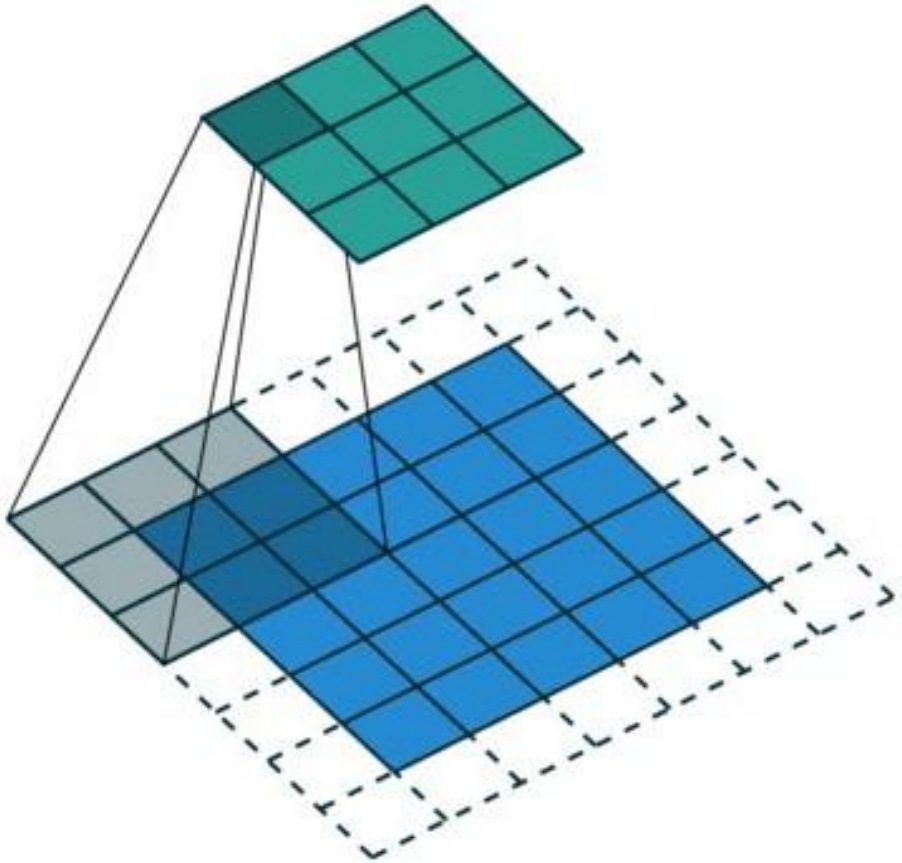
filter 

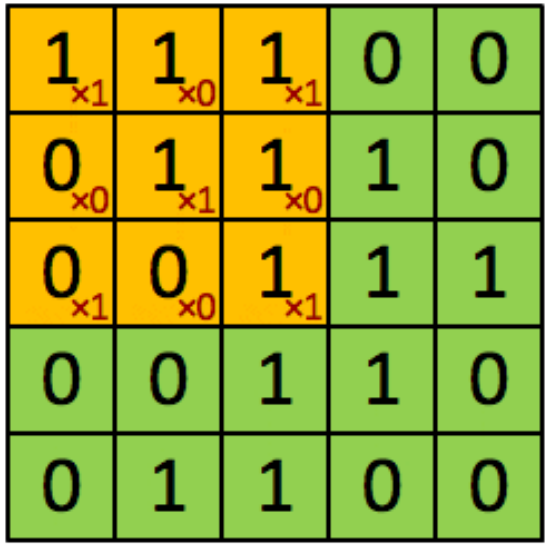
$$\mathbf{x}_{\mathbf{out}}[n,m] = b + \sum_{k,l=-N}^{N} \mathbf{w}[k,l]\mathbf{x}_{\mathbf{in}}[n-k,m-l]$$

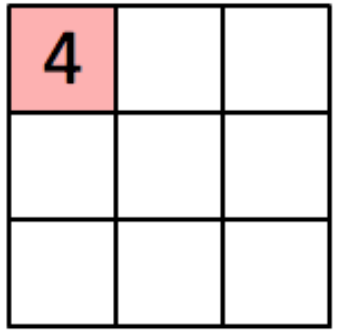
计算公式

卷积神经网络---卷积

卷积计算过程







卷积神经网络---卷积

卷积：填充（Padding）

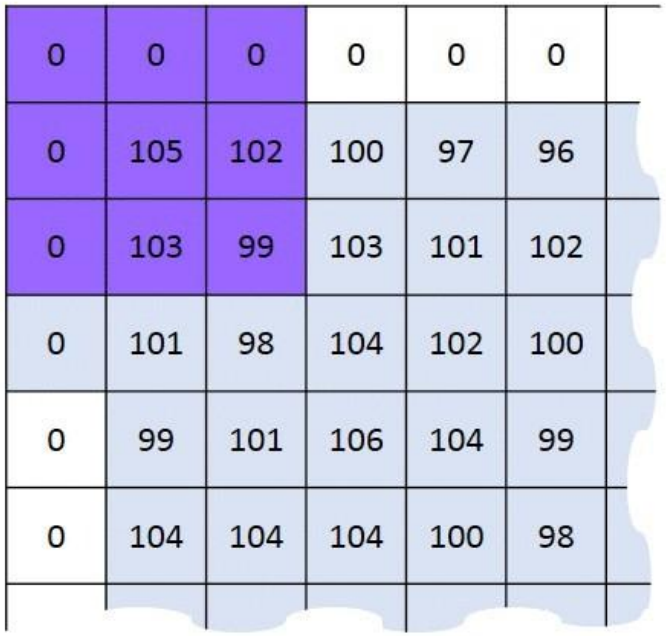
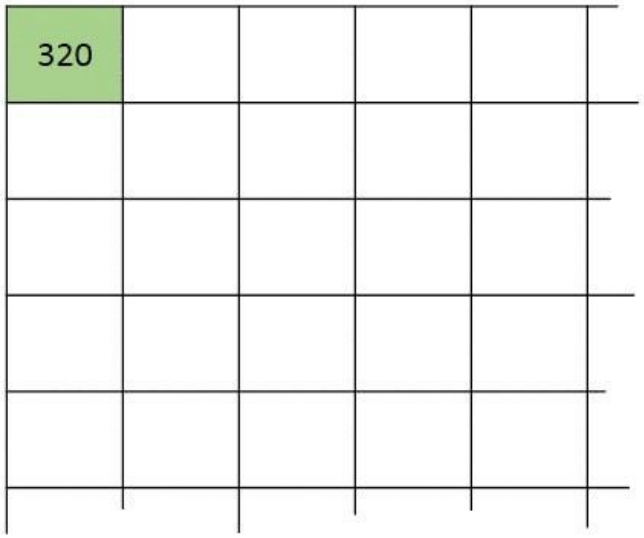


Image Matrix

Kernel Matrix

0	-1	0
-1	5	-1
0	-1	0



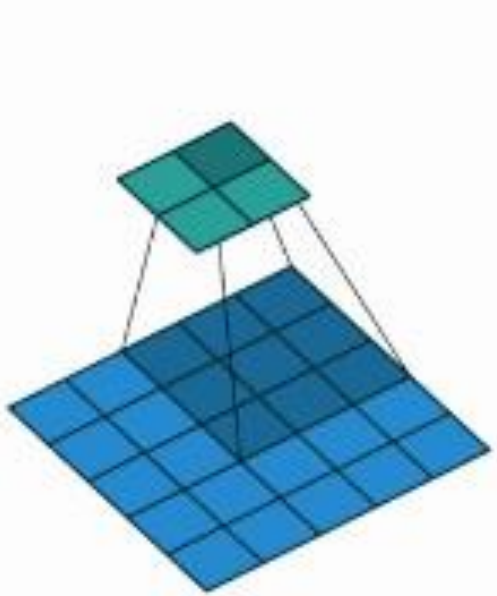
Output Matrix

$$0*0+0*-1+0*0$$
$$+0*-1+105*5+102*-1$$
$$+0*0+103*-1+99*0 = 320$$

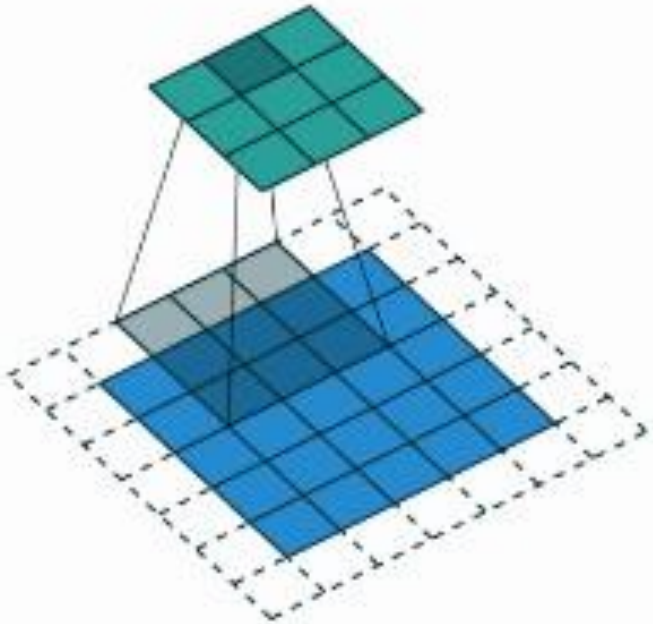
Convolution with horizontal and vertical strides = 1

卷积神经网络---卷积

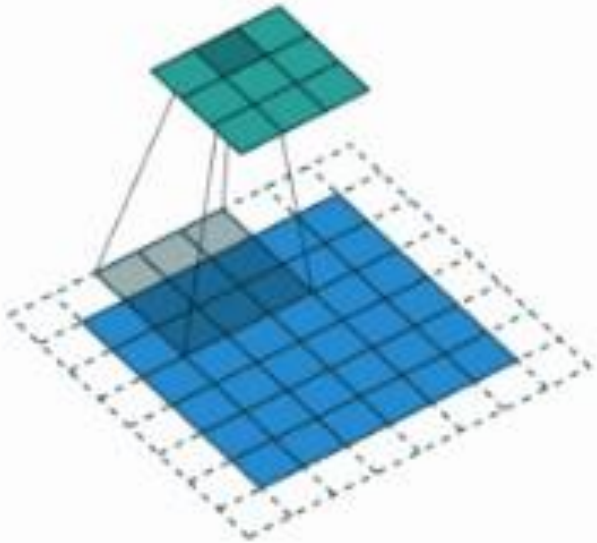
卷积：步长（Stride）



No padding, strides



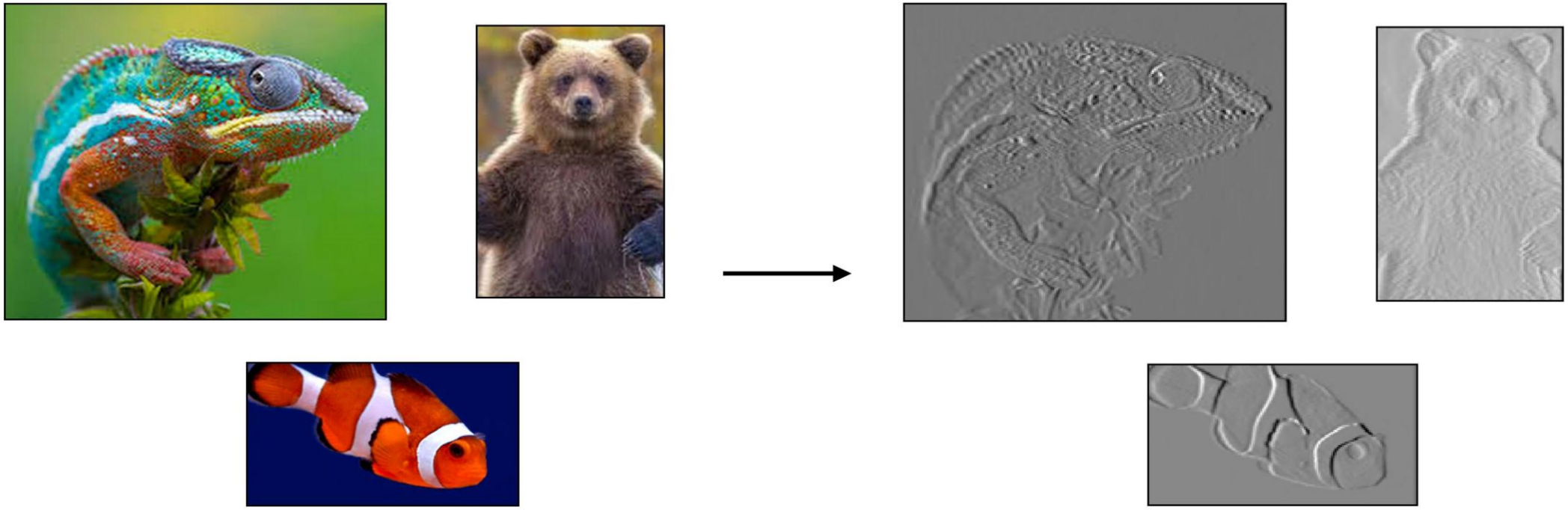
Padding, strides



Padding, strides (odd)

卷积神经网络---卷积

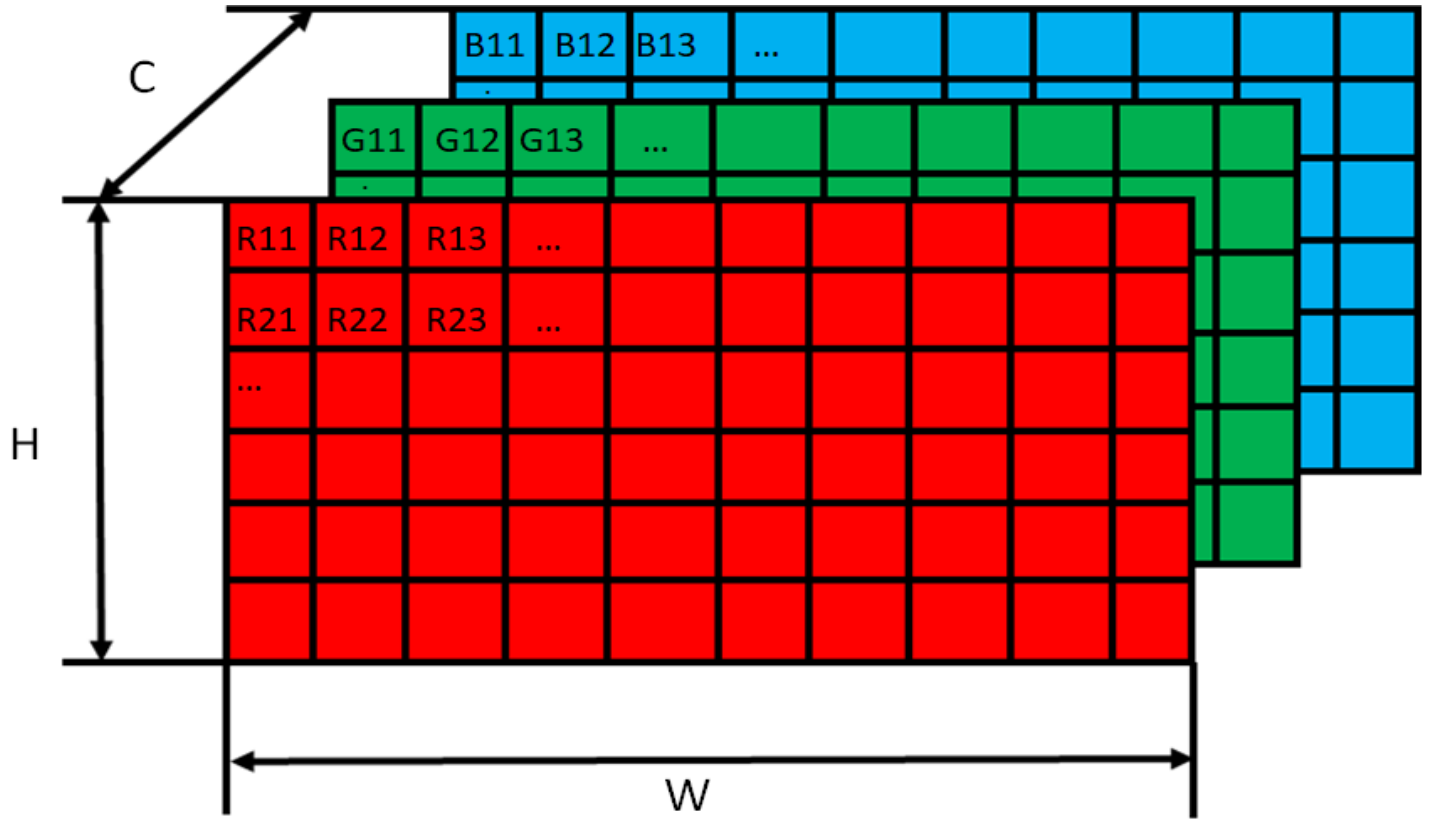
同样大小的卷积核可以应用于不同尺寸图像



Conv layers can be applied to arbitrarily-sized inputs

卷积神经网络---卷积

RGB图像格式

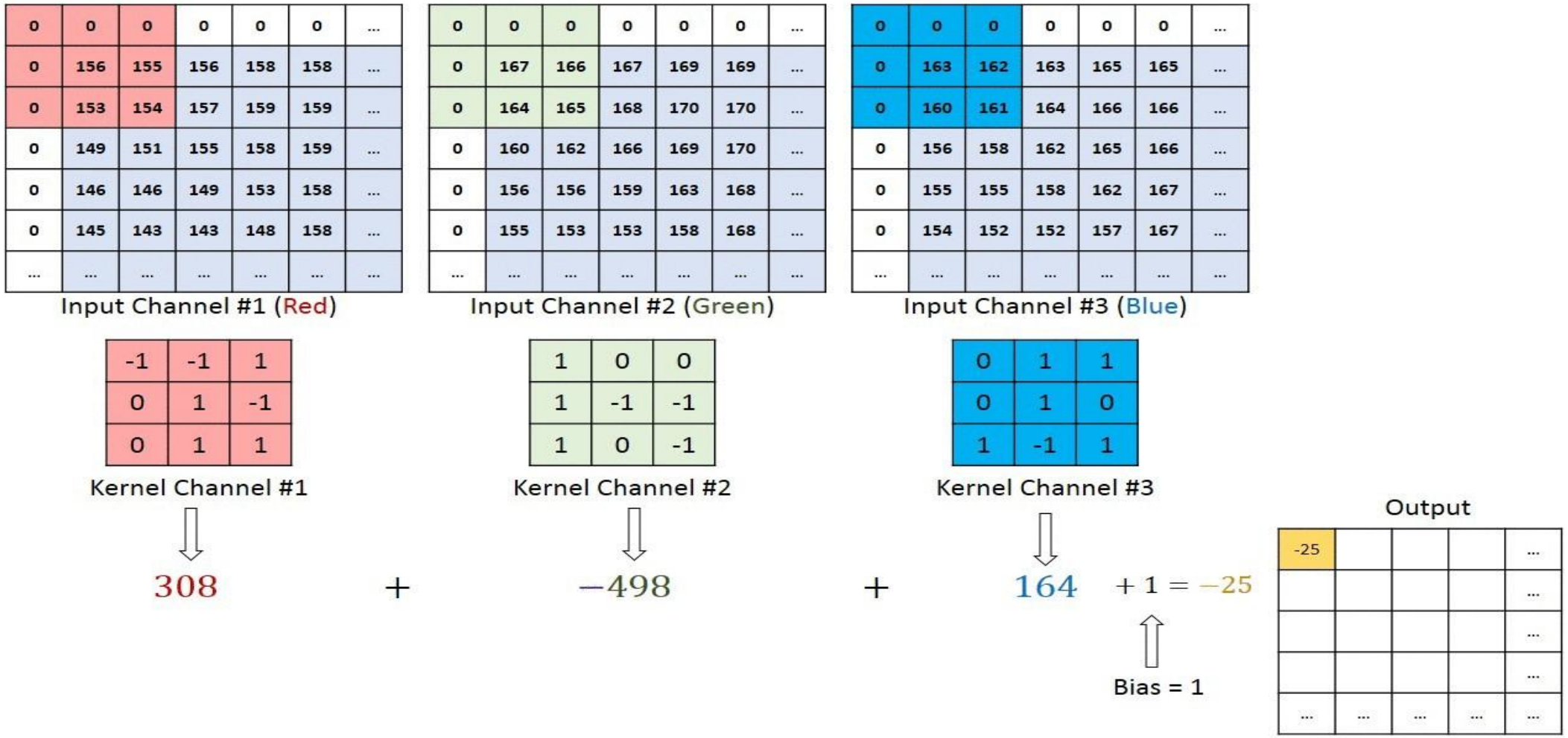


HWC：[R11 G11 B11 R12 G12 B21 ...]

CHW：[R11 R12 R13 ...R21 R22 R23 ... G11 G12 G13 ... G21 G22 G23 ... B11 B12 ...]

卷积神经网络---卷积

卷积算子：多输入通道&单输出通道



卷积神经网络---池化

- **最大池化（max pooling）：从输入特征图的某个区域子块中选择值最大的像素点作为最大池化结果（见图5.12）。**

- **平均池化（average pooling）：计算区域子块所包含所有像素点的均值，将均值作为平均池化结果。**

- **k-max池化（k-max pooling）：对输入特征图区域子块中的像素点取前k个最大值。如图5.13所示，从包含4个取值的每一列中选取前2个最大值就得到了k-max池化结果。**

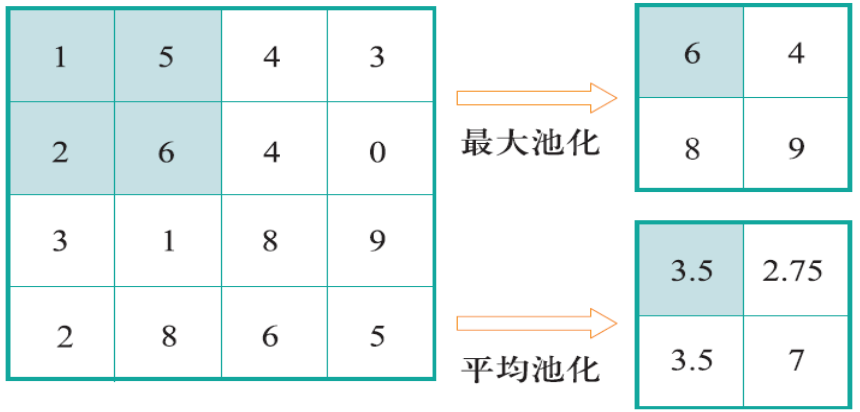


图 5.12　最大池化与平均池化操作示意

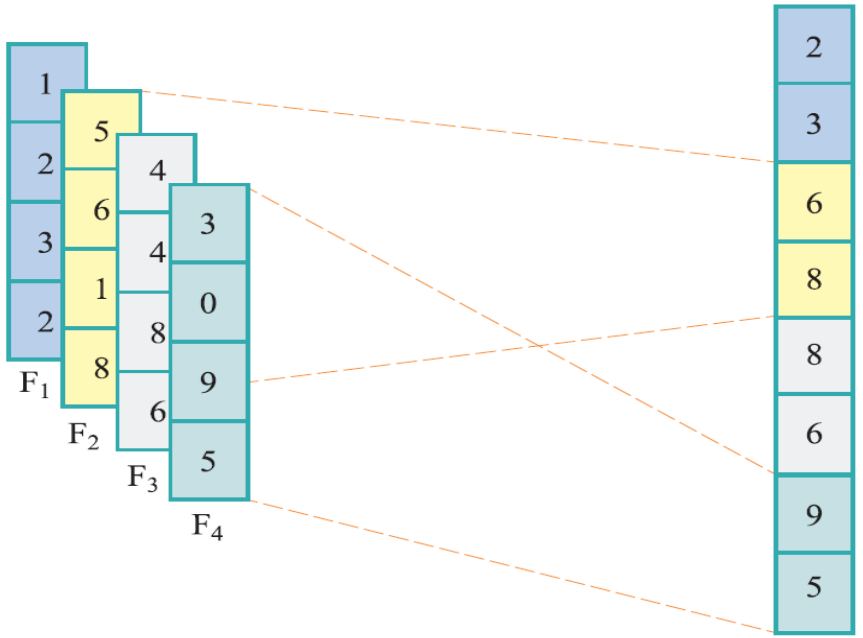


图 5.13　k–max 池化操作示意

卷积神经网络---网络结构

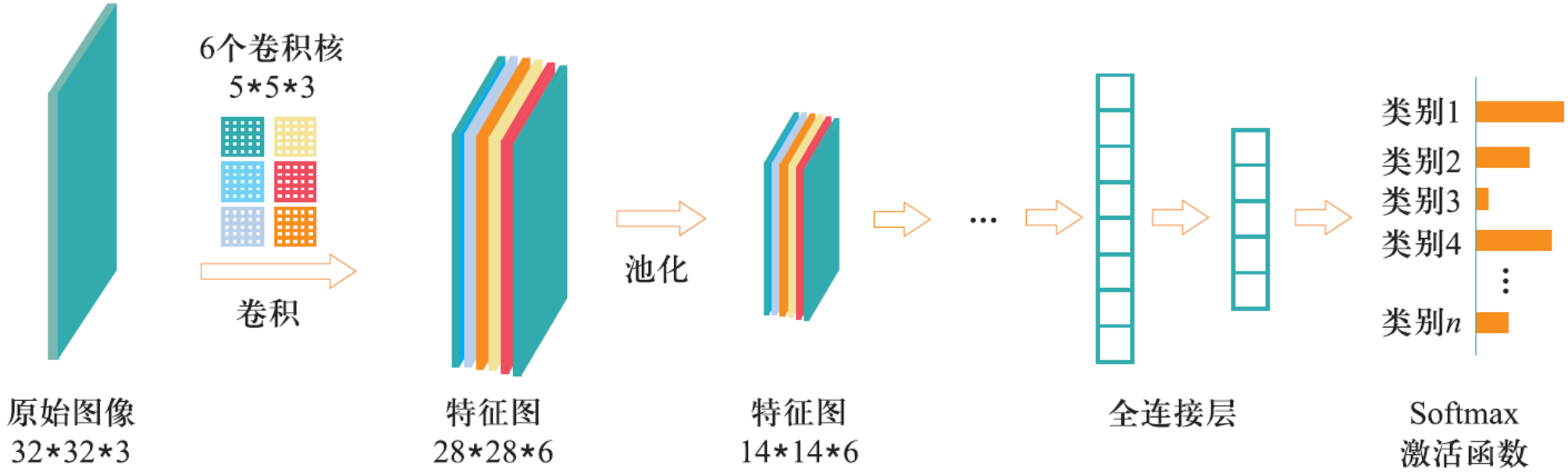
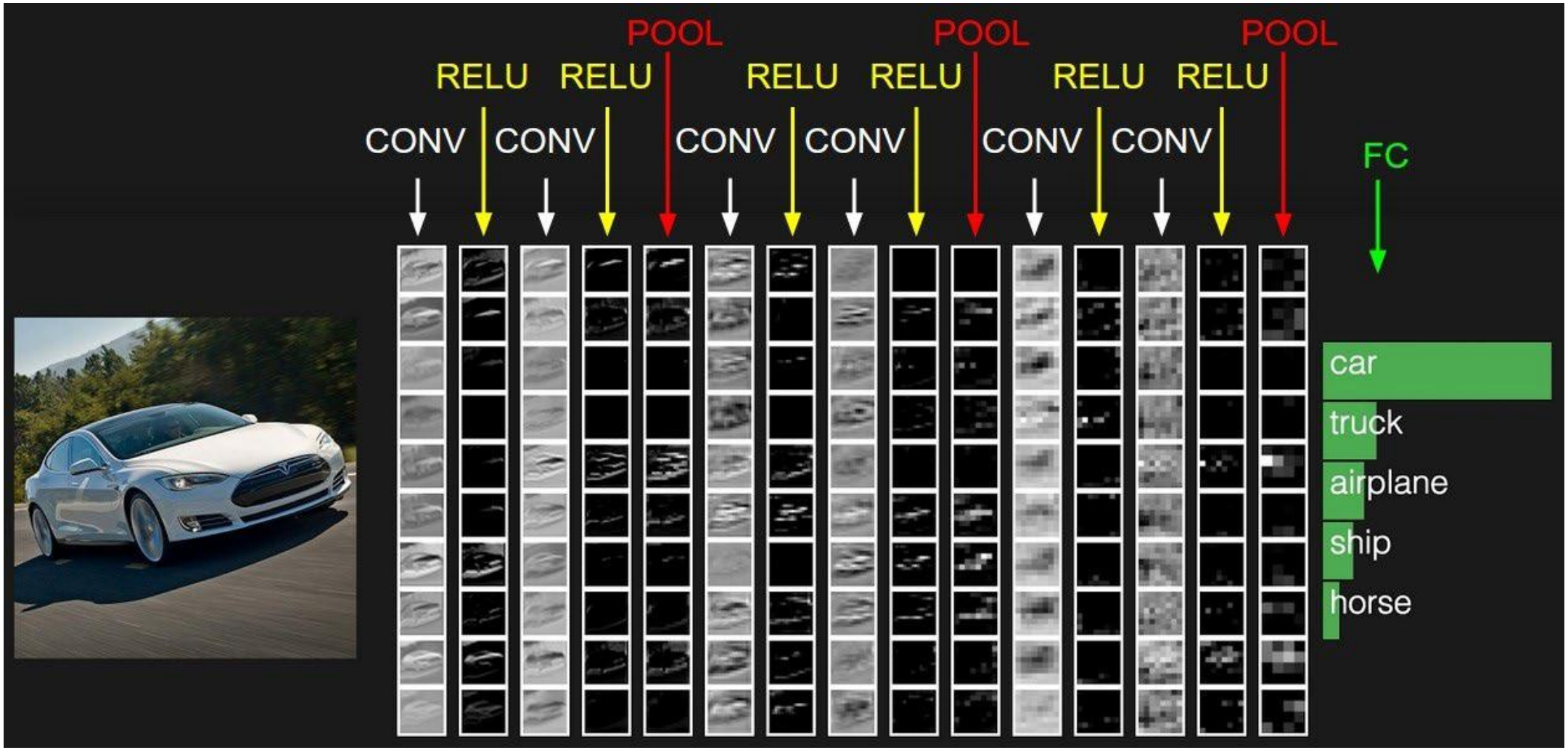


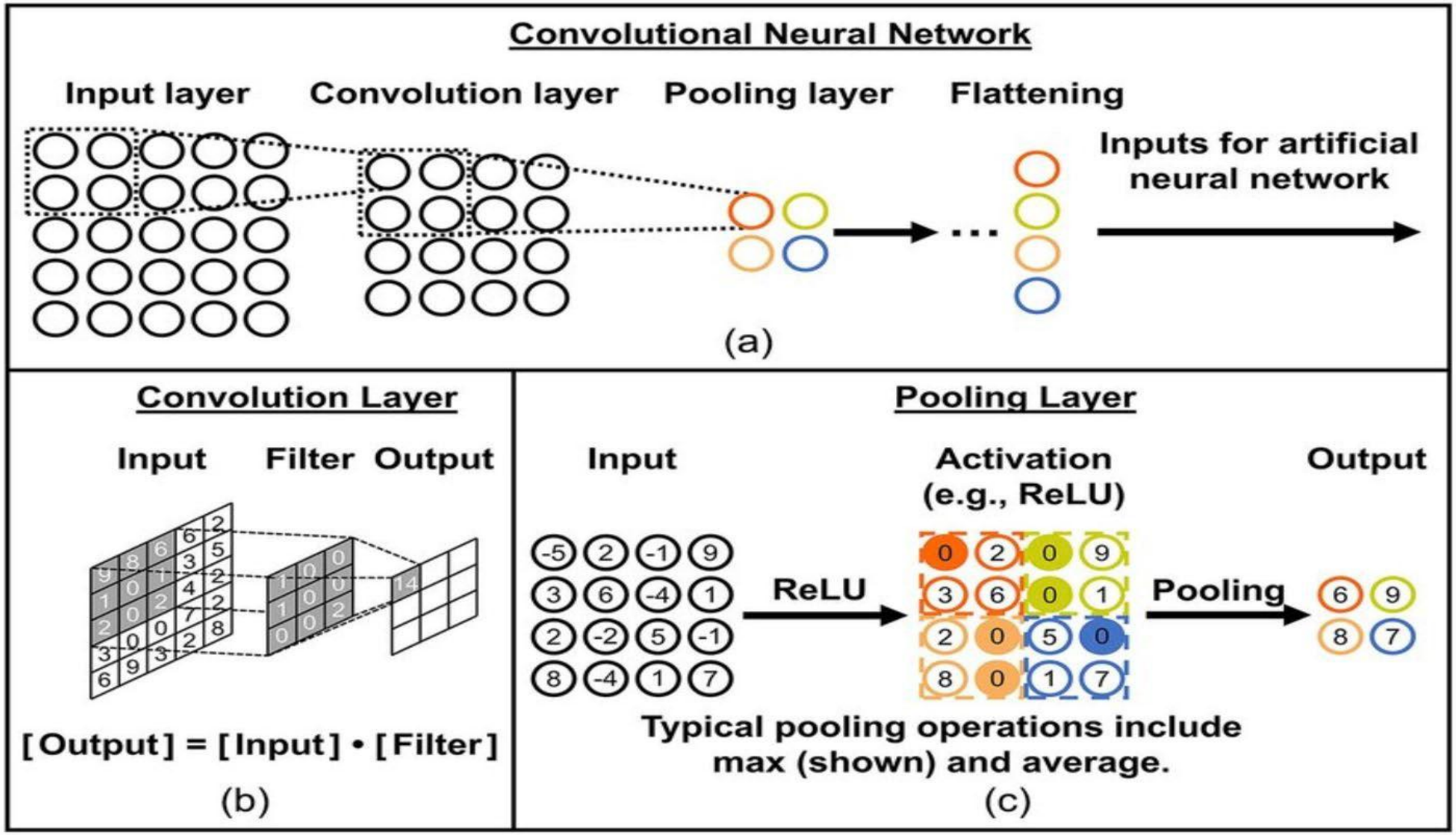
图 5.14　基于卷积神经网络的图像分类示意

如图5.14所示，对于32 × 32 × 3的彩色图像，先用6个5 × 5 × 3的卷积核对其进行卷积操作（步长为1，无填充），得到6个28 × 28的卷积结果（特征图），然后以2 × 2大小进行池化，得到6个14 × 14大小的特征图。最后通过若干个全连接层得到原始图像的向量表达，再对这个向量表达通过softmax函数进行分类识别，得到输入图像隶属于n个类别的概率。

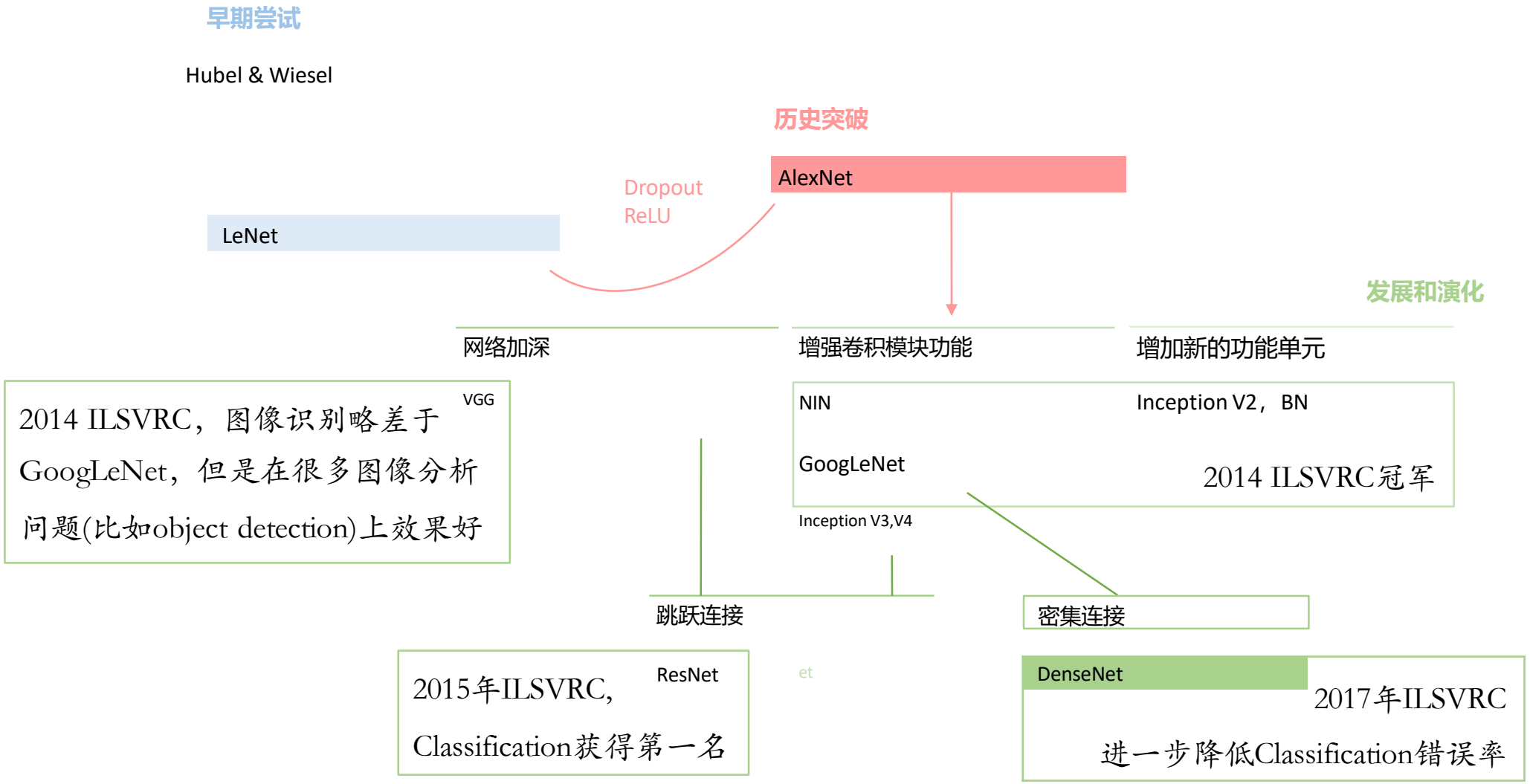
卷积神经网络---网络结构


POOL
POOL
POOL
RELU
RELU
RELU
RELU
RELU
RELU
CONV
CONV
CONV
CONV
CONV
CONV
FC
car
truck
airplane
ship
horse

卷积神经网络---网络结构

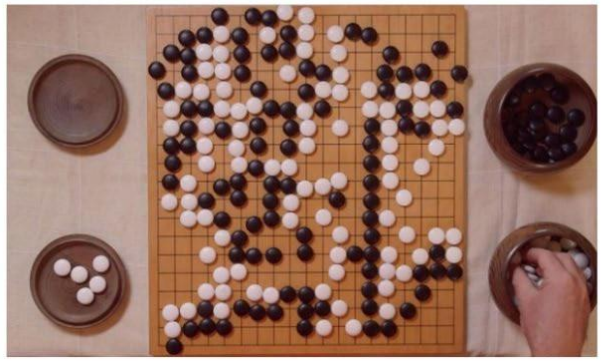

Convolutional Neural Network
Input layer
Convolution layer
Pooling layer
Flattening
Inputs for artificial neural network
(a)
Convolution Layer
Input
Filter
Output
[Output] = [Input] • [Filter]
(b)
Pooling Layer
Input
Activation (e.g., ReLU)
Output
ReLU
Pooling
Typical pooling operations include max (shown) and average.
(c)

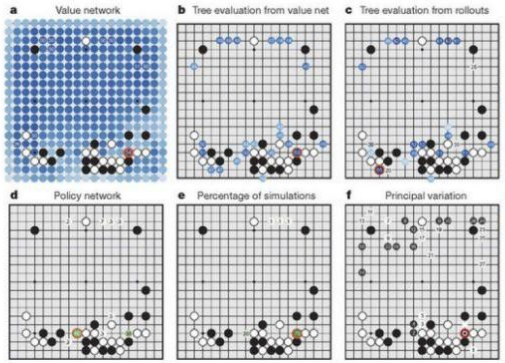
卷积神经网络---发展


早期尝试
Hubel & Wiesel
历史突破
AlexNet
Dropout
ReLU
LeNet
发展和演化
网络加深
增强卷积模块功能
增加新的功能单元
2014 ILSVRC，图像识别略差于GoogLeNet，但是在很多图像分析问题(比如object detection)上效果好
VGG
NIN
Inception V2，BN
GoogLeNet
2014 ILSVRC冠军
Inception V3,V4
跳跃连接
密集连接
2015年ILSVRC，Classification获得第一名
ResNet
et
DenseNet
2017年ILSVRC
进一步降低Classification错误率

卷积神经网络---应用

AlphaGo







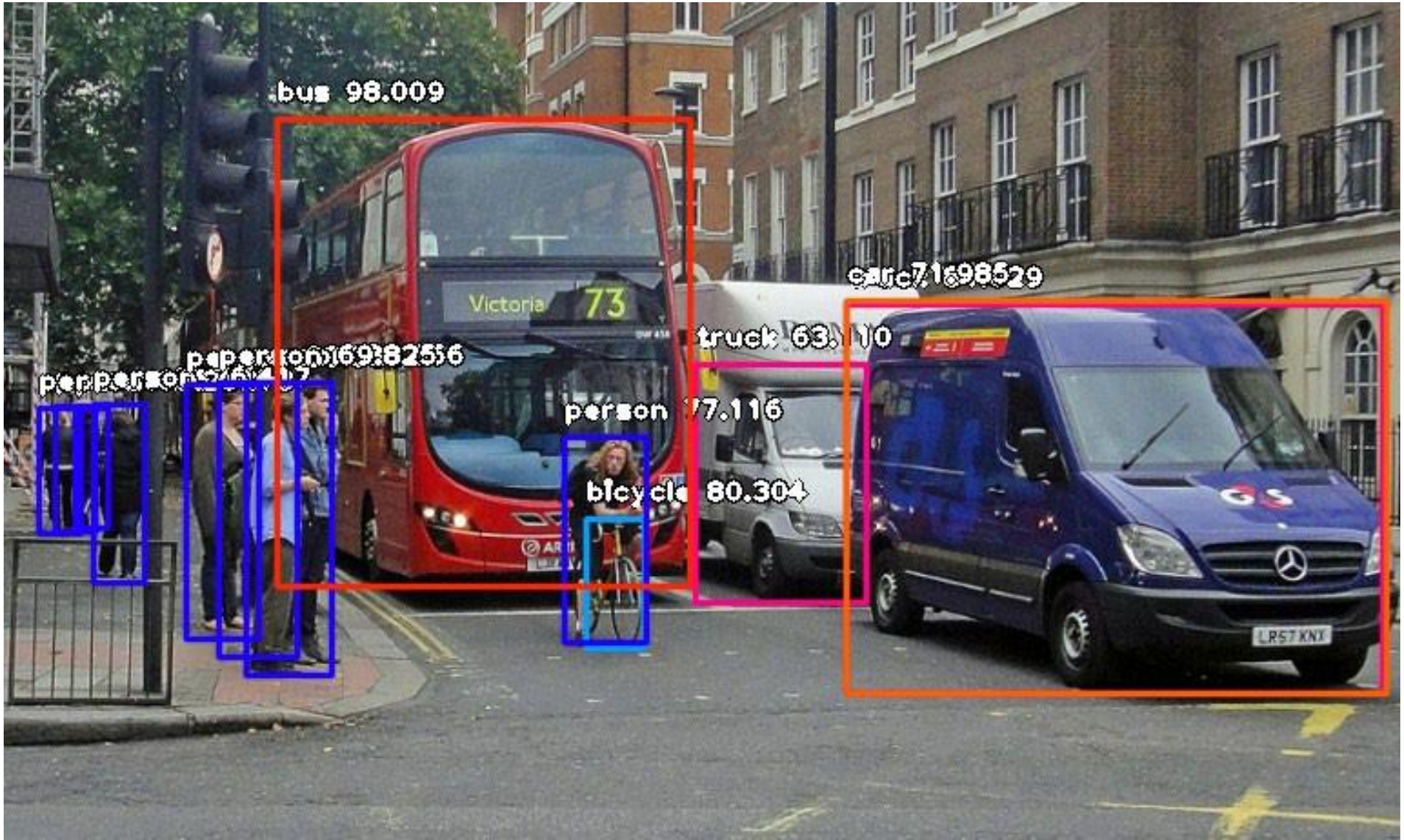
The input to the policy network is a 19 × 19 × 48 image stack consisting of 48 feature planes. The first hidden layer zero pads the input into a 23 × 23 image, then convolves *k* filters of kernel size 5 × 5 with stride 1 with the input image and applies a rectifier nonlinearity. Each of the subsequent hidden layers 2 to 12 zero pads the respective previous hidden layer into a 21 × 21 image, then convolves *k* filters of kernel size 3 × 3 with stride 1, again followed by a rectifier nonlinearity. The final layer convolves 1 filter of kernel size 1 × 1 with stride 1, with a different bias for each position, and applies a softmax function. The match version of AlphaGo used *k* = 192 filters; Fig. 2b and Extended Data Table 3 additionally show the results of training with *k* = 128, 256 and 384 filters.

policy network:
[19x19x48] Input
CONV1: 192 5x5 filters , stride 1, pad 2 => [19x19x192]
CONV2..12: 192 3x3 filters, stride 1, pad 1 => [19x19x192]
CONV: 1 1x1 filter, stride 1, pad 0 => [19x19] *(probability map of promising moves)*

分布式系统：1202 个CPU 和176 块GPU
单机版：48 个CPU 和8 块GPU
走子速度：3 毫秒-2 微秒

卷积神经网络---应用

目标检测（Object Detection）



卷积神经网络---应用

Mask RCNN实例分割



Figure 4. More results of **Mask R-CNN** on COCO test images, using ResNet-101-FPN and running at 5 fps, with 35.7 mask AP (Table 1).

卷积神经网络---应用

OCR文字识别



卷积神经网络---应用

图像生成



卷积神经网络---应用

Deep Dream超现实的视觉效果



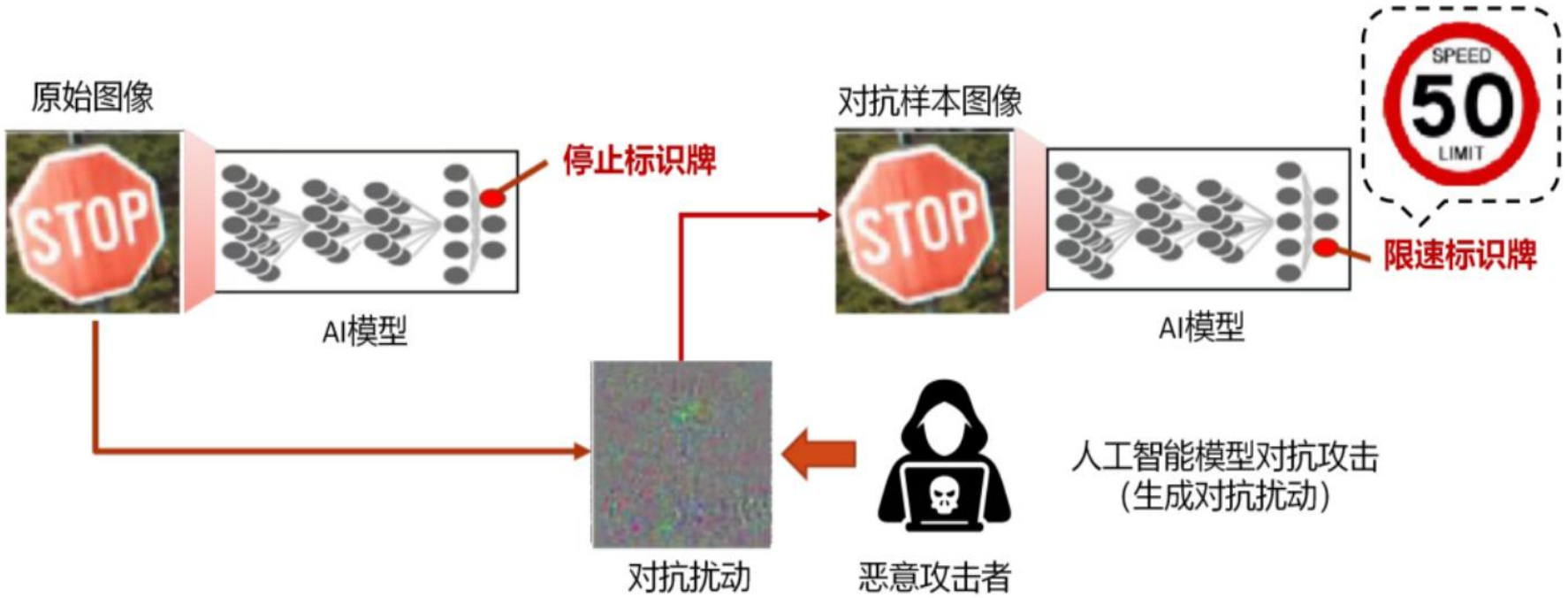
卷积神经网络---应用

画风迁移



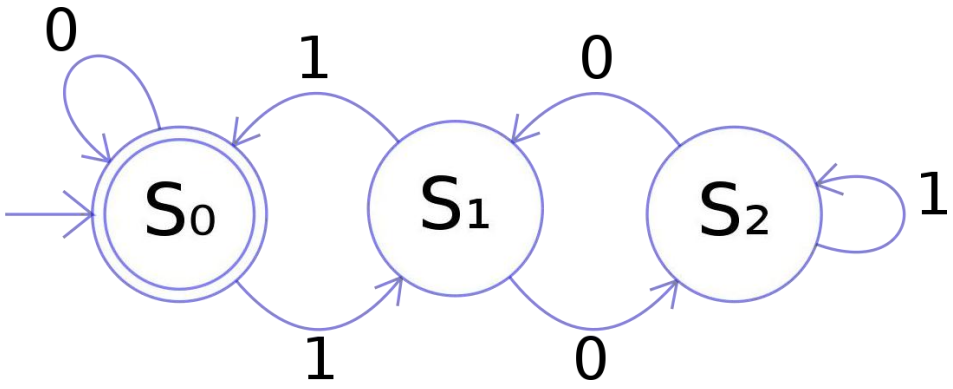
卷积神经网络---应用

对抗样本



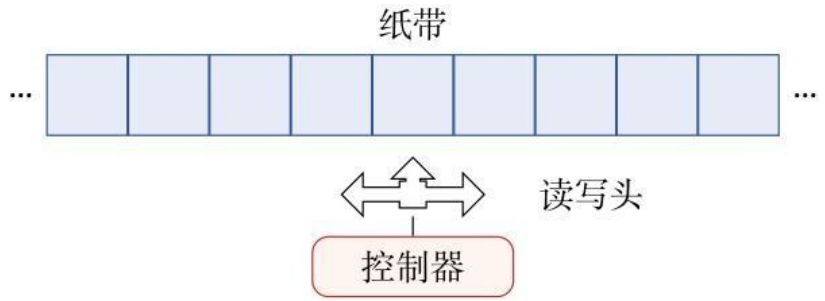
循环神经网络---引入

有限状态自动机（Finite Automata)



图灵机

- 一种抽象数学模型，可以用来模拟任何可计算问题。



需要记忆能力

如何给网络增加记忆能力？

循环神经网络---概念

- 循环神经网络（Recurrent Neural Network, RNN）是处理序列数据（如文本句子、视频帧等）时所采用的网络结构。前面的前馈神经网络或卷积神经网络所需要处理的输入数据一次性给定，难以处理存在前后依赖关系的数据。
- 1990年，Jeffrey L.Elman提出循环神经网络的最初概念框架，循环神经网络的本质是希望模拟人所具有的记忆能力，在学习过程中记住部分已经出现的信息，并利用所记住的信息影响后续结点输出。
- 循环神经网络在自然语言处理，例如语音识别、情感分析、机器翻译等领域有重要应用。

循环神经网络---概念

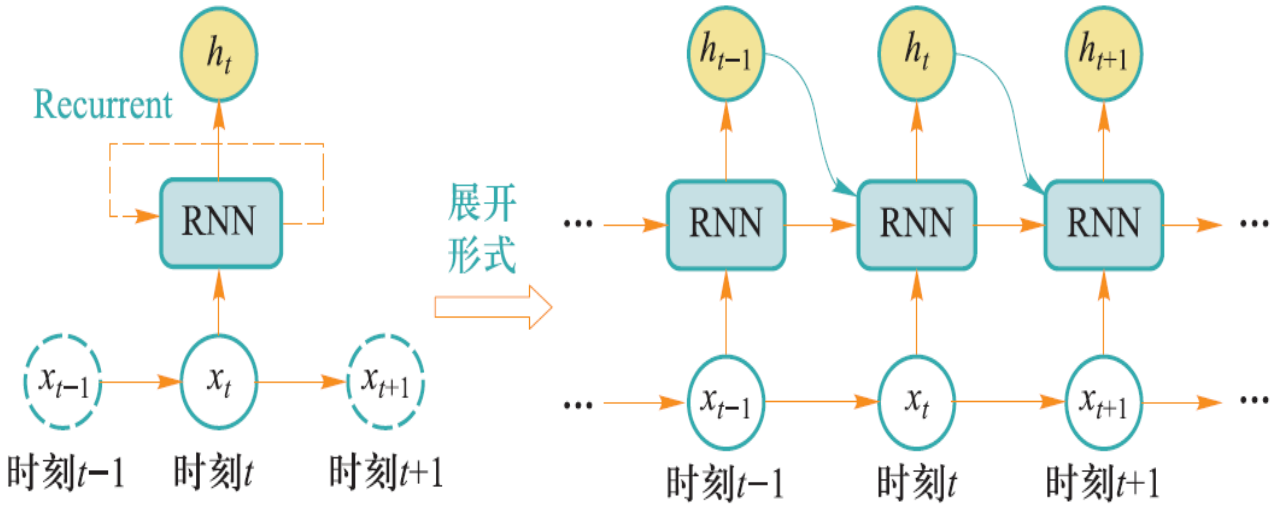


图 5.15　循环神经网络结构及其展开形式示意

图5.15 循环神经网络的结构图。对于一个序列数据，在每一时刻t，循环神经网络单元会读取当前输入数据x_t和前一时刻输入数据x_{t-1}所对应的隐式编码结果h_{t-1}，一起生成t时刻的隐式编码结果h_t。接着将h_t后传，去参与生成$t+1$时刻输入数据x_{t+1}的隐式编码h_{t+1}。如此循环处理，直至该序列数据被处理完毕。

隐式编码h_{t-1}已经“记忆”了t时刻之前的时序信息

循环神经网络---概念

在时刻t，一旦得到当前输入数据x_t，循环神经网络会结合前一时刻$t-1$得到的隐式编码h_{t-1}，如下产生当前时刻隐式编码h_t：

$$h_t = \Phi(\mathbf{U} \times x_t + \mathbf{W} \times h_{t-1})$$

这里$\Phi(\cdot)$是激活函数，一般可为Sigmoid或者Tanh激活函数，U与W为模型参数。从这里可看出，当前时刻的隐式编码输出h_t不仅仅与当前输入数据x_t相关，与网络已有的“记忆”h_{t-1}也有着密不可分的联系。

循环神经网络---处理自然语言的例子

- 以自然语言例子来解释循环神经网络是如何处理时序数据

  将自然语言句子进行分类，分类为饮食、体育、文化等不同类别。

  (a) 对文本语句进行分词成单词，(b) 读入每个单词后，产生对应单词的隐式编码，并利用其来参与后续单词的隐式编码。(c)考虑到句子中所有单词的信息均被后向传递，因此将最后一个单词的输出作为整个句子向量编码表示；(d)将每个单词的隐式编码进行加权平均，将加权平均结果作为句子的向量编码表示。这样，一旦得到了句子的向量编码表示，就可以利用某个分类器，对句子的类别进行判断。

循环神经网络---处理自然语言的例子

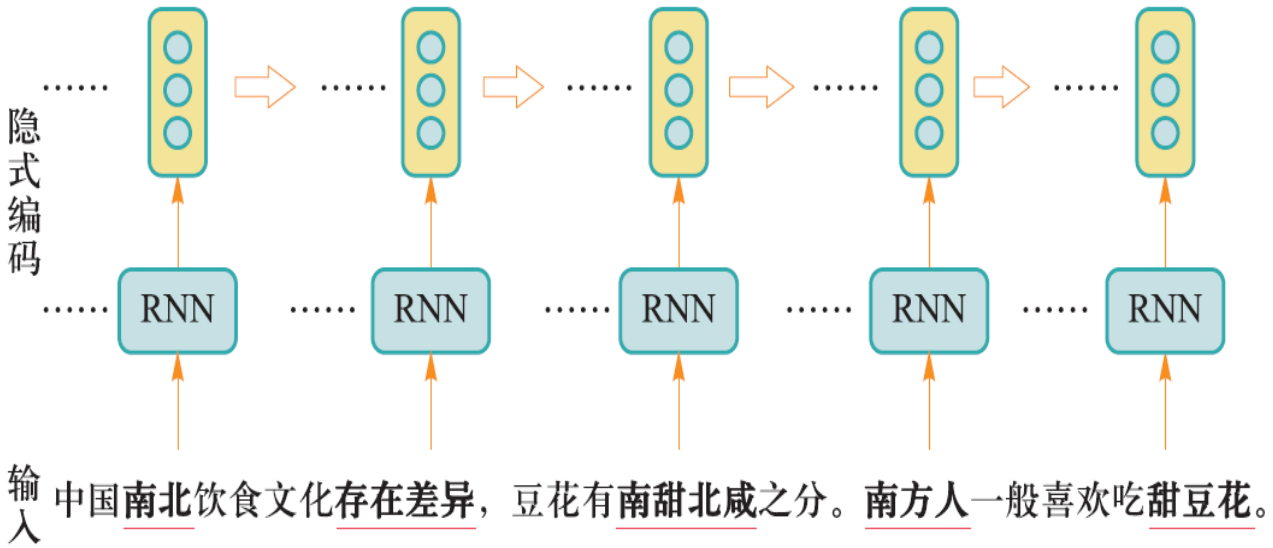


图 5.16　循环神经网络的应用示例

“南北”“存在差异”“南甜北咸”“南方人”“甜豆花”等单词之间存在前后依赖，这种依赖会被有效利用起来，得到各单词的隐式编码及句子的向量表达。

循环神经网络---应用模式

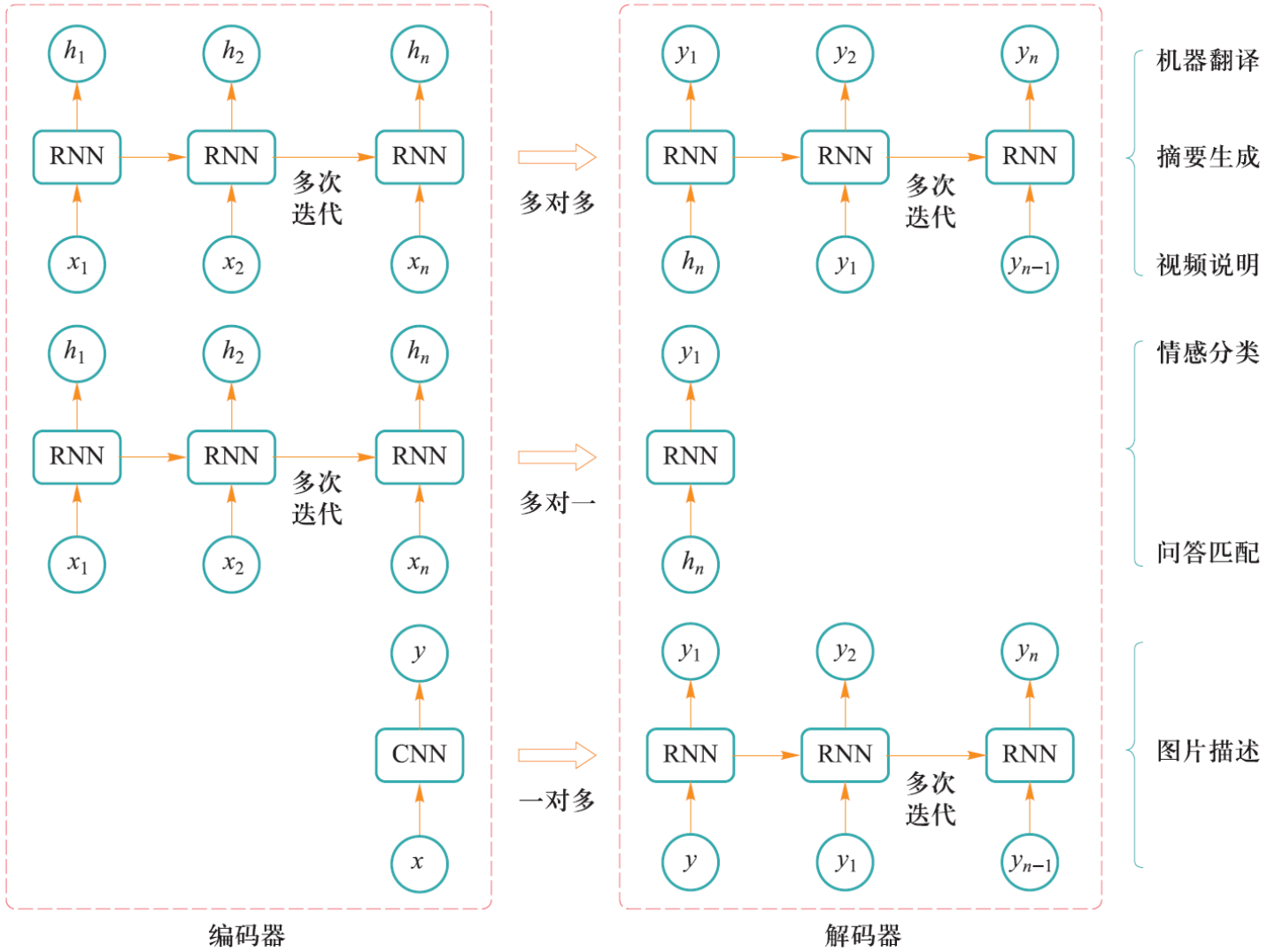


图 5.17　输入输出不同情况下循环神经网络结构示例

循环神经网络---梯度传递

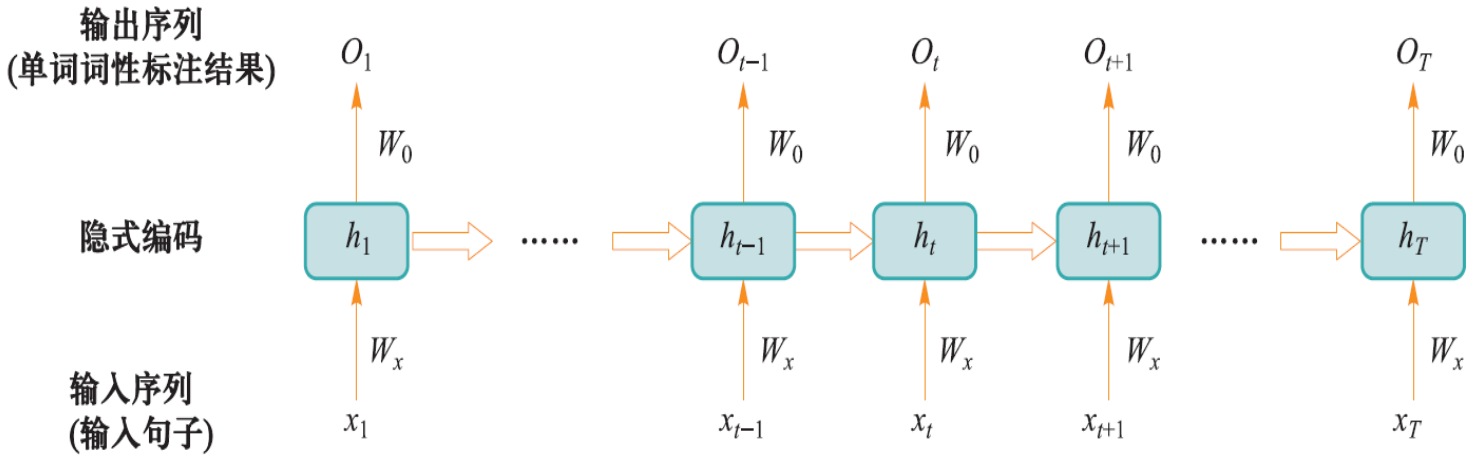


图 5.18　循环神经网络中梯度传递示意

- 图5.18以词性标注为例子给出了一个输入序列数据$(x_1, \cdots, x_{t-1}, x_t, x_{t+1}, \ldots, x_T)$被循环神经网络处理的示意图。时刻$t$输入数据$x_t$（文本中的单词）的隐式编码为$h_t$、其真实输出为$y_t$（单词词性）、模型预测$x_t$的词性是$O_t$。参数$W_x$将$x_t$映射为隐式编码$h_t$、参数$W_o$将$h_t$映射为预测输出$O_t$、$h_{t-1}$通过参数$W_h$参与$h_t$的生成。图中$W_x$、$W_o$和$W_h$是复用参数。

- 假设时刻t隐式编码如下得到：$h_t = \tanh(W_x x_t + W_h h_{t-1} + b)$。使用交叉熵损失函数计算时刻$t$预测输出与实际输出的误差$E_t$。显然，整个序列产生的误差为$E = \frac{1}{2}\sum_{t=1}^{T} E_t$。

循环神经网络---梯度传递

下面介绍如何根据时刻t所得误差来更新参数W_x。在时刻t计算所得O_t不仅涉及到了时刻t的W_x，而且也涉及了前面所有时刻的W_x，按照链式求导法则，E_t在对W_x求导时候，也需要对前面时刻的W_x依次求导，然后再将求导结果进行累加，即：

$$\frac{\partial E_t}{\partial W_x} = \sum_{i=1}^{t} \frac{\partial E_t}{\partial O_t} \frac{\partial O_t}{\partial h_t} \left(\prod_{j=i+1}^{t} \frac{\partial h_j}{\partial h_{j-1}} \right) \frac{\partial h_i}{\partial W_x}$$

其中$\prod_{j=i+1}^{t} \frac{\partial h_j}{\partial h_{j-1}} = \prod_{j=i+1}^{t} tanh' \times W_h$

令$t = 3$，则有：

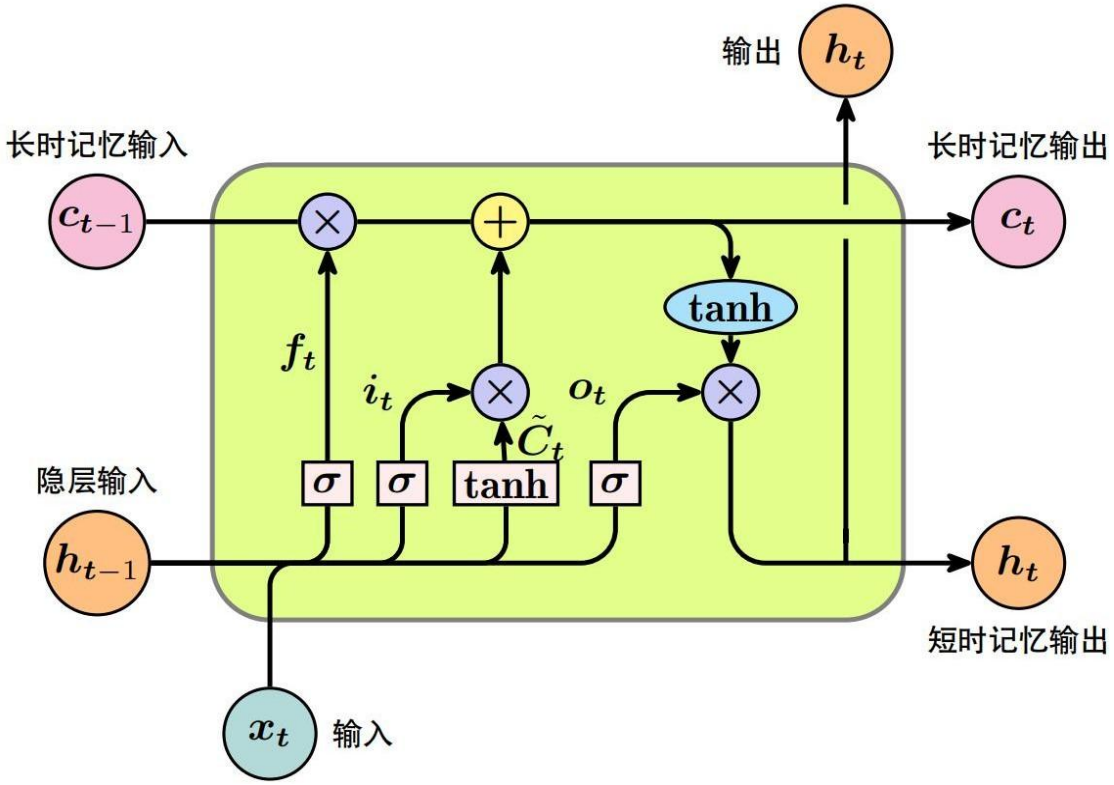
$$\frac{\partial E_3}{\partial W_x} = \frac{\partial E_3}{\partial O_3} \frac{\partial O_3}{\partial h_3} \frac{\partial h_3}{\partial W_x} + \frac{\partial E_3}{\partial O_3} \frac{\partial O_3}{\partial h_3} \frac{\partial h_3}{\partial h_2} \frac{\partial h_2}{\partial W_x} + \frac{\partial E_3}{\partial O_3} \frac{\partial O_3}{\partial h_3} \frac{\partial h_3}{\partial h_2} \frac{\partial h_2}{\partial h_1} \frac{\partial h_1}{\partial W_x}$$

由于$tanh$函数的导数取值位于0到1区间，对于长序列而言，若干多个0到1区间的小数相乘，会使得参数求导结果很小，引发梯度消失问题。E_t对W_h的求导类似，这里就不列出了。为了缓解梯度消失问题，长短时记忆模型（Long Short-Term Memory，LSTM）被提出。

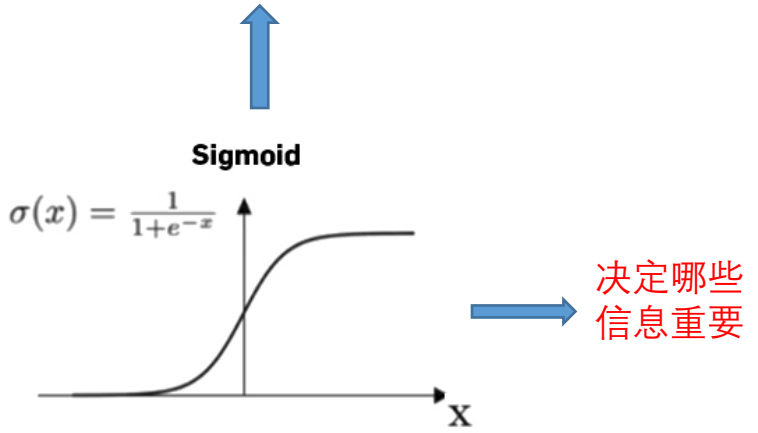
循环神经网络---长短时记忆模型

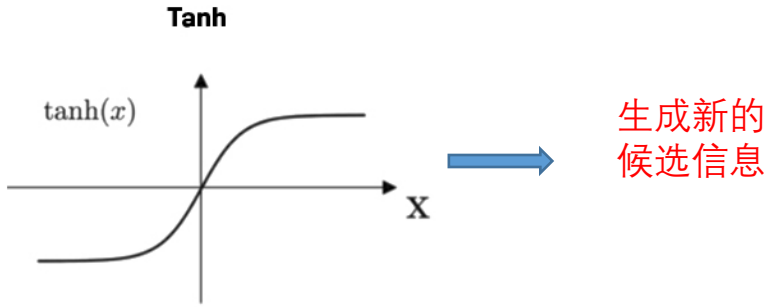
与其默认遗忘，不如学习何时应该遗忘

—— Jürgen Schmidhuber （LSTM之父）



三种门结构





循环神经网络---长短时记忆模型

- 与简单的循环神经网络结构不同，长短时记忆网络中引入了内部记忆单元（internal memory cell）和门（gates）结构来对当前时刻输入信息以及前序时刻所生成信息进行整合和传递。在这里，内部记忆单元中信息可视为对“历史信息”的累积。
- LSTM模型中有输入门(input gate)、遗忘门(forget gate)和输出门(output gate)三种门结构。对于给定的当前时刻输入数据x_t和前一时刻隐式编码h_{t-1}，输入门、遗忘门和输出门通过各自参数对其编码，分别得到三种门结构的输出i_t、f_t和o_t。
- 在此基础上，再进一步结合前一时刻内部记忆单元信息c_{t-1}来更新当前时刻内部记忆单元信息c_t，最终得到当前时刻的隐式编码h_t

循环神经网络---长短时记忆模型

- Long Short-Term Memory, LSTM
  - 记忆单元（Cell State）：存储和访问信息，并通过门控机制 来控制信息的遗忘和记忆
  - 遗忘门、输入门和输出门

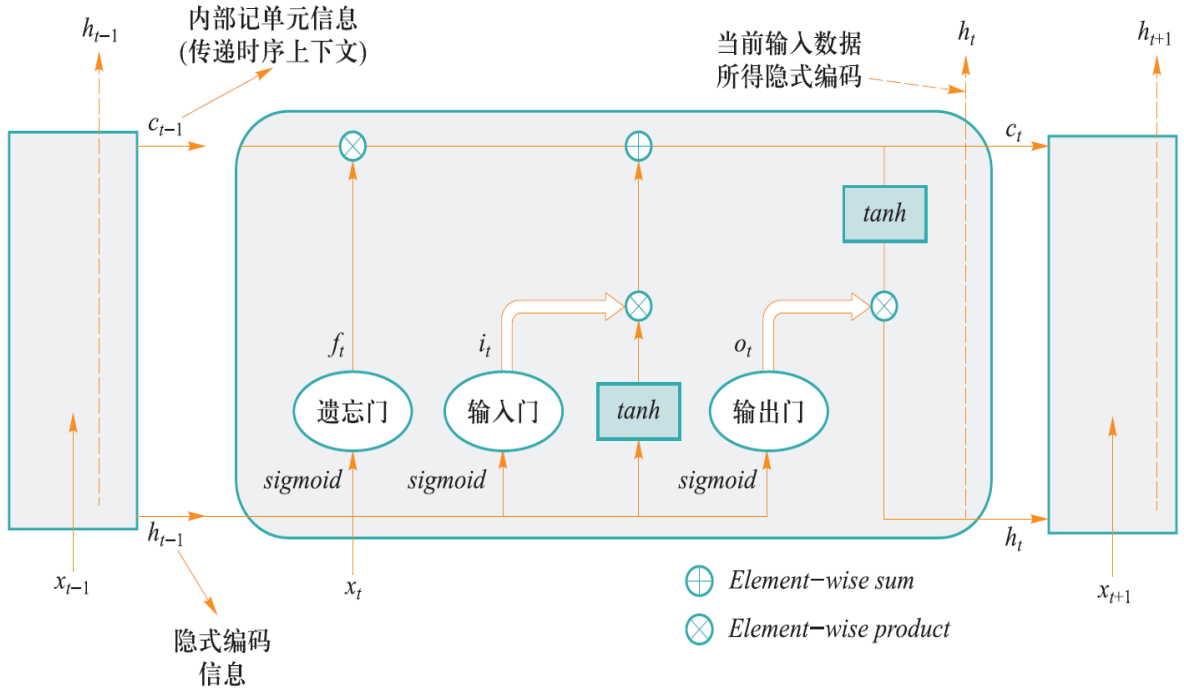


图 5.19　长短时记忆网络模型

（注：在每个时刻 t，只有内部记忆单元信息 c_t 和隐式编码 h_t 这两种信息起到了对序列信息进行传递的作用；只有 h_t 作为本时刻的输出用于分类等处理）

$$\mathbf{i}_t = \sigma(W_i\mathbf{x}_t + U_i\mathbf{h}_{t-1} + \mathbf{b}_i), \qquad \tilde{\mathbf{c}}_t = \tanh(W_c\mathbf{x}_t + U_c\mathbf{h}_{t-1} + \mathbf{b}_c)$$

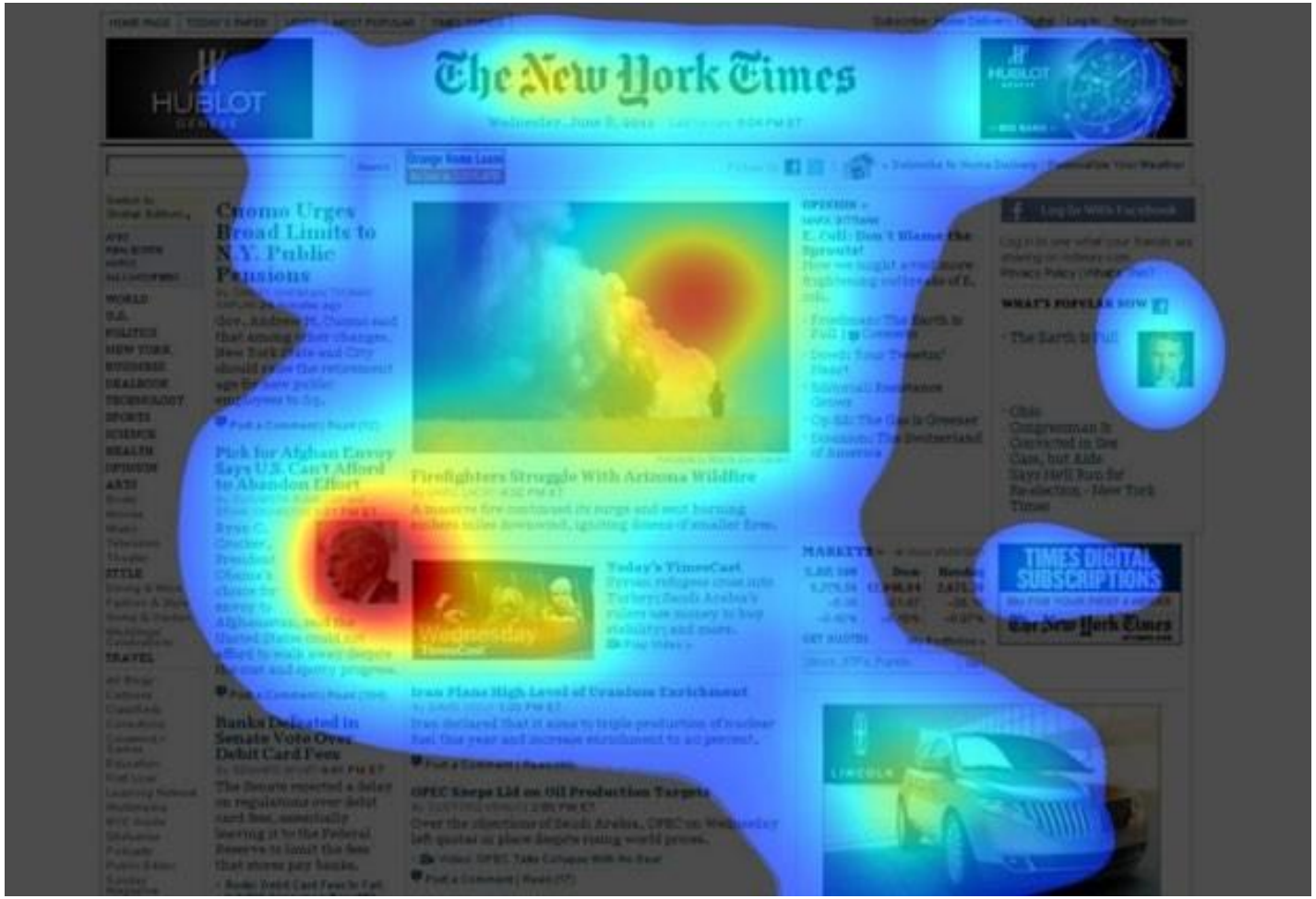
$$\mathbf{f}_t = \sigma(W_f\mathbf{x}_t + U_f\mathbf{h}_{t-1} + \mathbf{b}_f), \qquad \mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t,$$

$$\mathbf{o}_t = \sigma(W_o\mathbf{x}_t + U_o\mathbf{h}_{t-1} + \mathbf{b}_o), \qquad \mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t),$$

注意力机制与正则化---大脑中的注意力

- 人脑每个时刻接收的外界输入信息非常多，包括来源于视觉、听 觉、触觉的各种各样的信息。
- 但就视觉来说，眼睛每秒钟都会发送千万比特的信息给视觉神经 系统。
- 人脑通过注意力来解决信息超载问题。

注意力机制与正则化---注意力示例

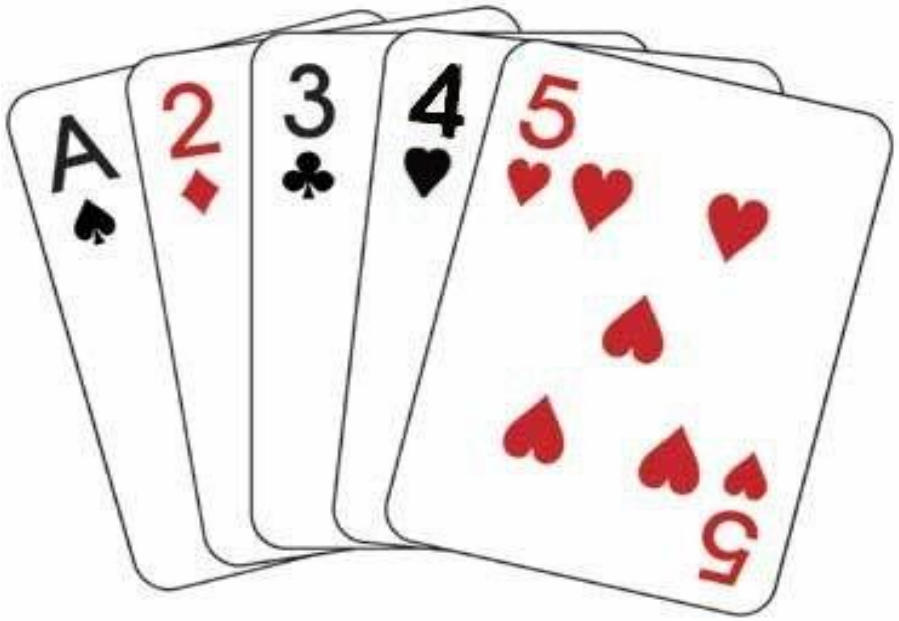


注意力机制与正则化---注意力示例

- 当一个人在吵闹的鸡尾酒会上和朋友聊天时，尽管周围噪音干扰 很多，他还是可以听到朋友的谈话内容，而忽略其他人的声音。
- 同时，如果未注意到的背景声中有重要的词（比如他的名字）， 他会马上注意到。

鸡尾酒会效应　　考试?

注意力机制与正则化---注意力示例



所有黑色牌的数字之和为多少?

注意力机制与正则化---生物学中的注意力

- 非自主性提示：基于环境中物体的突出性和易见性
- 自主性提示：认知和意识的控制

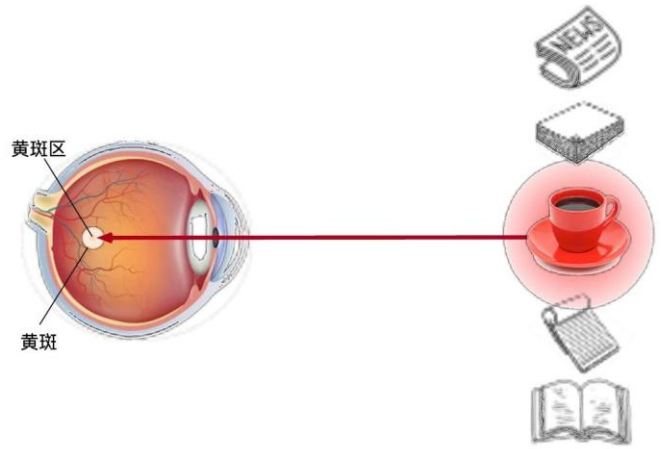


图10.1.1: 由于突出性的非自主性提示（红杯子），注意力不自主地指向了咖啡杯

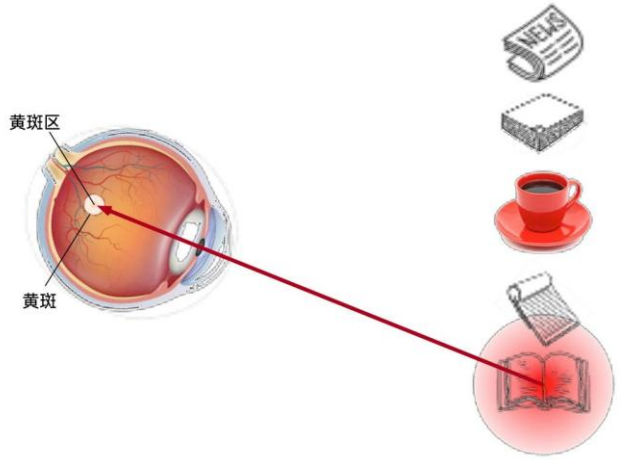


图10.1.2: 依赖于任务的意志提示（想读一本书），注意力被自主引导到书上

汇聚（pooling）
自下而上

MaxPooling?

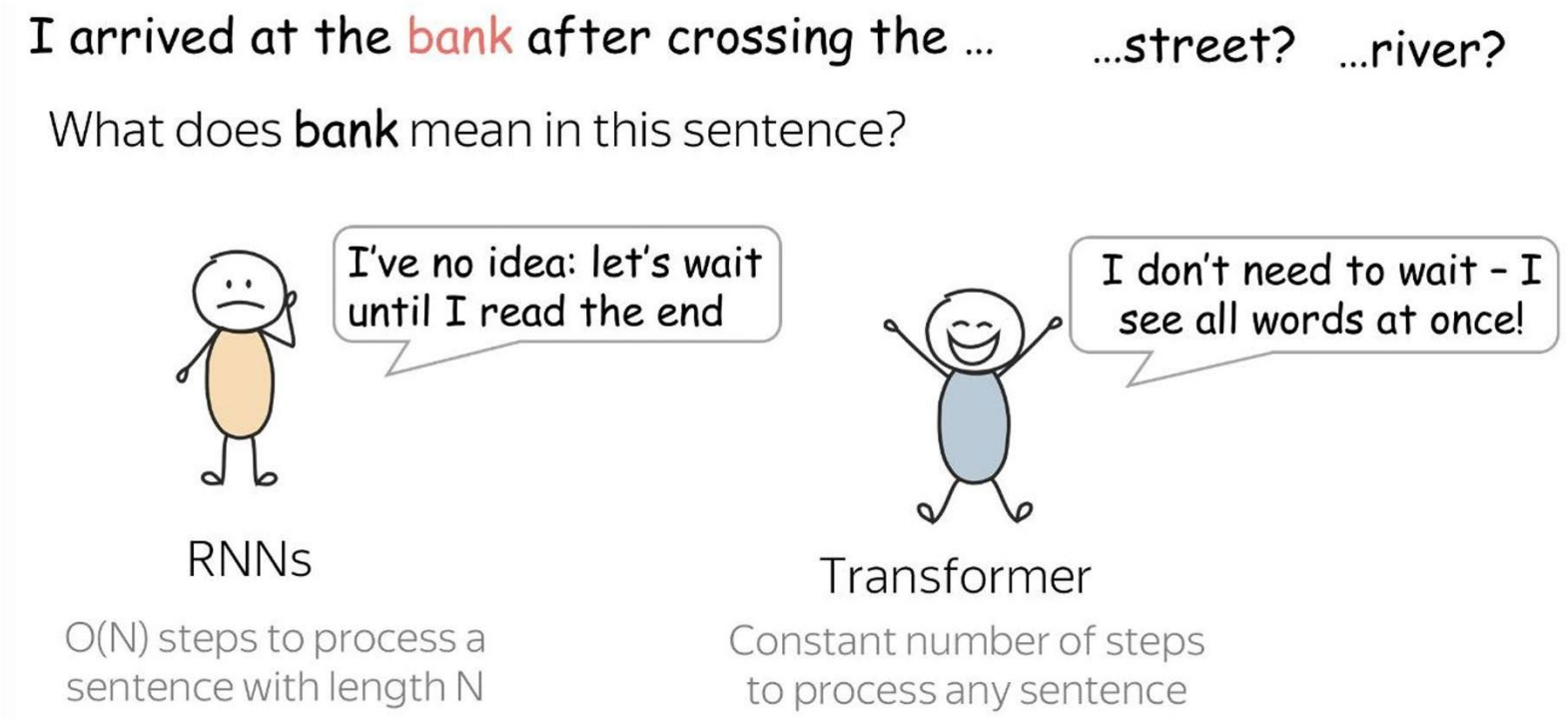
聚焦（focus）
自上而下

注意力机制与正则化---人工神经网络中的注意力机制

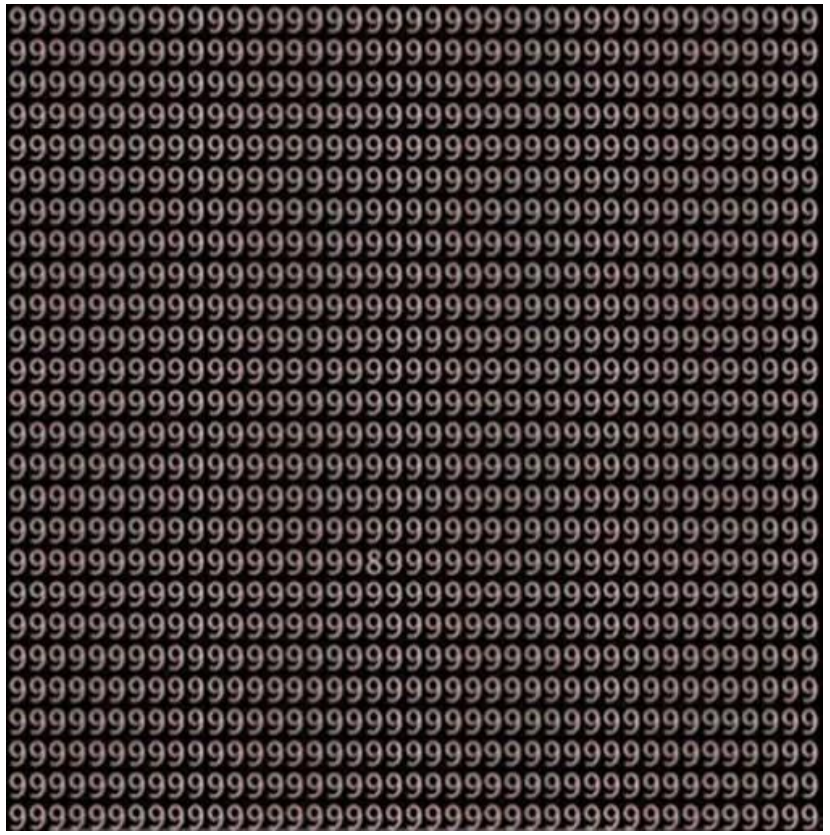
- 注意力，“全局信息查询”

Transformer vs. RNN

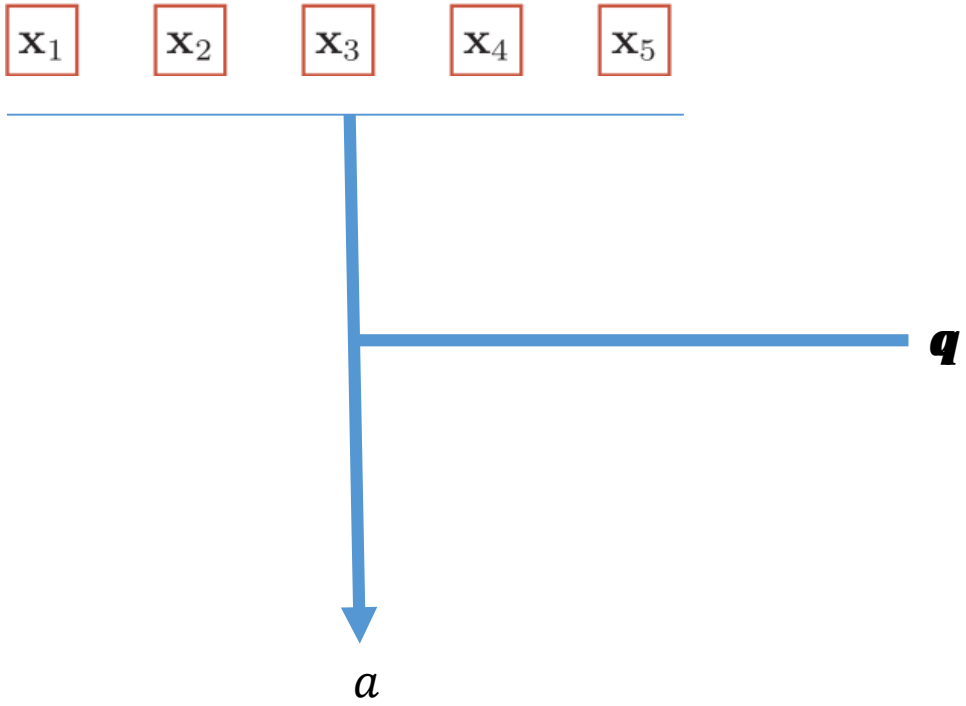
- Transformer：一眼看见所有单词
- RNN：序列读入一个一个单词



注意力机制与正则化---如何实现注意力机制?



Try to find 8 less than 8 seconds

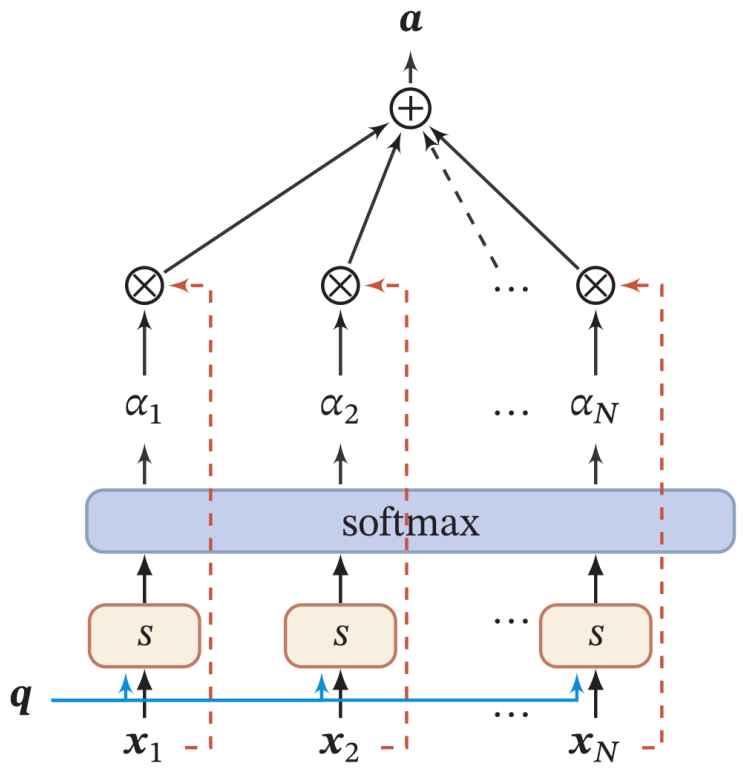


注意力机制与正则化---注意力机制

- 软性注意力机制（soft attention mechanism）
- 注意力机制可以分为两步
  - 计算注意力分布α

$$\alpha_n = p(z = n|\boldsymbol{X}, \boldsymbol{q}) = \text{softmax}\left(s(\boldsymbol{x}_n, \boldsymbol{q})\right) = \frac{\exp\left(s(\boldsymbol{x}_n, \boldsymbol{q})\right)}{\sum_{j=1}^{N} \exp\left(s(\boldsymbol{x}_j, \boldsymbol{q})\right)}$$

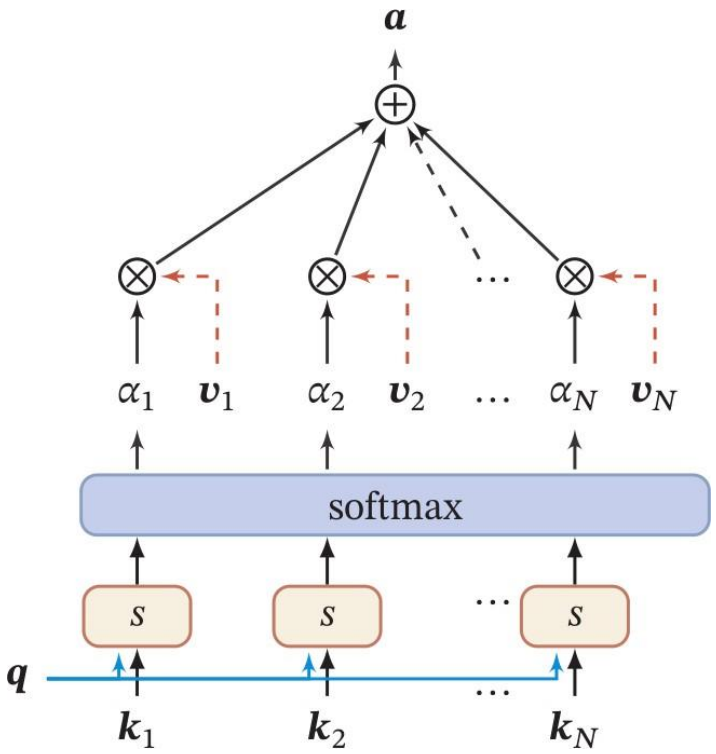
$$\text{att}(\boldsymbol{X}, \boldsymbol{q}) = \sum_{n=1}^{N} \alpha_n \boldsymbol{x}_n, = \mathbb{E}_{z\sim p(z|\boldsymbol{X},\boldsymbol{q})}[\boldsymbol{x}_z]$$



s(x_n,q)打分函数, q是query输入一个查询

- 根据α来计算输入信息的加权平均。

注意力机制与正则化---注意力机制



用 $(K,V)=[(k_1,v_1),\cdots(k_N,v_N)]$ 表示N个输入信息

$$\text{att}\Big((\boldsymbol{K},\boldsymbol{V}),\boldsymbol{q}\Big)=\sum_{n=1}^{N}\alpha_n\boldsymbol{v}_n,$$

$$=\sum_{n=1}^{N}\frac{\exp\Big(s(\boldsymbol{k}_n,\boldsymbol{q})\Big)}{\sum_j\exp\Big(s(\boldsymbol{k}_j,\boldsymbol{q})\Big)}\boldsymbol{v}_n$$

- Q是查询(query)，表示要寻找的目标（**搜索物品单词**），K是键(key)，表示与查询匹配，确定相关性（**被搜索物品的描述信息**），V是值（value）表示与键对应的实际数据内容（**搜索物品的语义内容**）
- 把Q和K各建模成一个向量，Q和K计算一个相似度（比如向量内积） s(k_n,q)相似度函数

注意力机制与正则化---Transformer自注意力

- 首先生成每个单词的内嵌向量（包含单词在句子中位置编码向量信息）
- 生成每个单词w_i的查询向量q_i（query）、键向量k_i (key)和值向量v_i (value)：
- $q_i = W^q \times w_i$；$k_i = W^k \times w_i$；$v_i = W^v \times w_i$；W^q、W^k和W^v三个映射矩阵是自注意力模型需要训练的全部参数。
- 计算给定单词和句子中其他单词之间归一化关联度（softmax()函数的输出），利用关联度大小来对每个单词的自身内容调整和加权取和，求出每个单词在句子中因为上下文语义而与其他单词之间的自注意力。

注意力机制与正则化---Transformer自注意力

1、计算w_3所对应查询向量与其他单词健向量之间的关联度（点积）： $\alpha_{3i} = q_3 \cdot k_i$

2、对α_{3i}通过softmax函数进行归一化，得到a'_{3i}，a'_{3i}为“意在”单词与其他单词所具有的重要程度（即注意力）。

3、a'_{3i}乘以每个单词w_i所对应的值向量v_i，即$a'_{3i} \times v_i$。

4、对上一步结果进行$\sum_j a'_{3i} \times v_i$，这个结果作为在当前句子语境下单词$w_3$“注意”到与其他单词的关联程度（self-attention）。

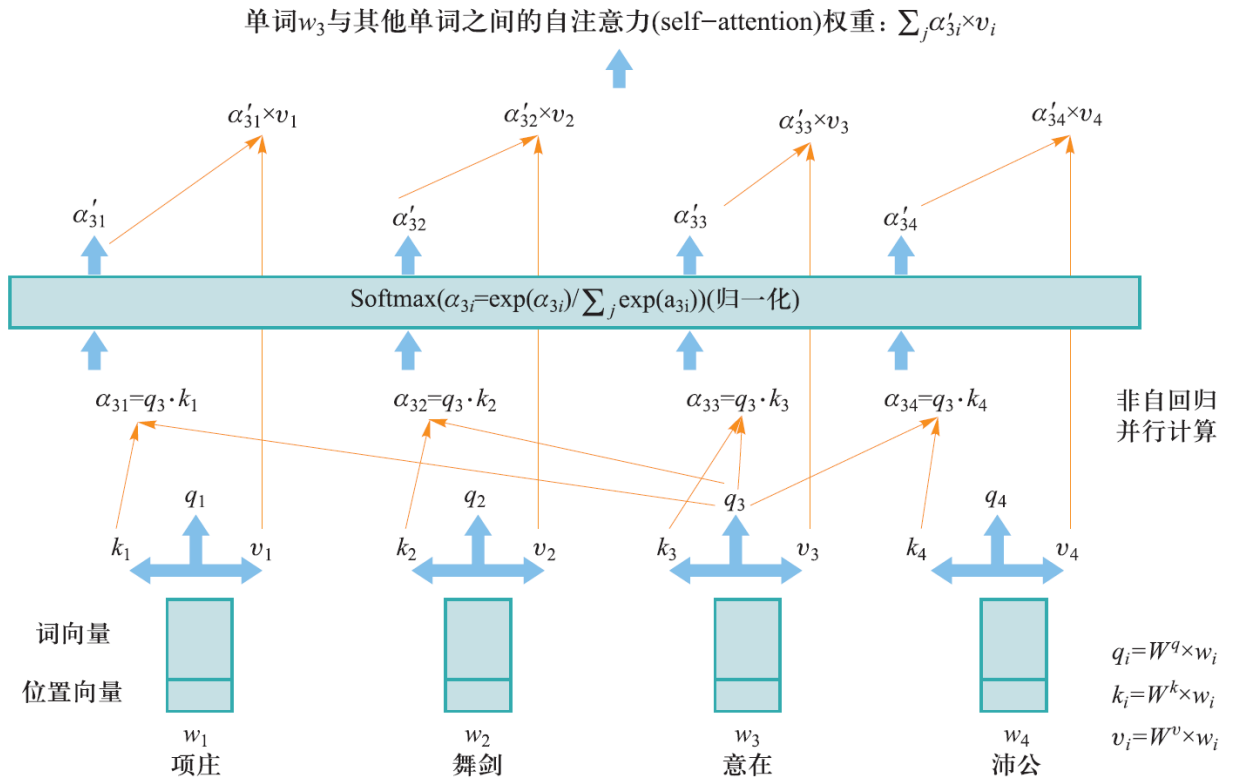


图 5.21　句子中每个单词与其他单词自注意力取值计算示意（以单词 w_3 为例）

注意力机制与正则化---正则化

深度神经网络结构复杂、参数众多，很容易造成过拟合（over-fitting）。为了缓解神经网络在训练过程中出现的过拟合现象，需要采取一些正则化技术来提升神经网络的泛化能力（generalization）。

- **Dropout：指在训练神经网络的过程中随机丢掉（屏蔽）一部分神经元来降低神经网络的复杂度，从而防止过拟合。**
- **批归一化：批归一化（batch normalization）就是通过规范化手段，把神经网络每层中任意神经元的输入值分布改变成均值为0、方差为1的标准正态分布，把偏移较大的分布强制映射为标准正态分布。经过批归一化处理，激活函数的输入值被映射到非线性函数梯度较大的区域，使得梯度变大从而克服梯度消失问题，进而加快收敛速度。**

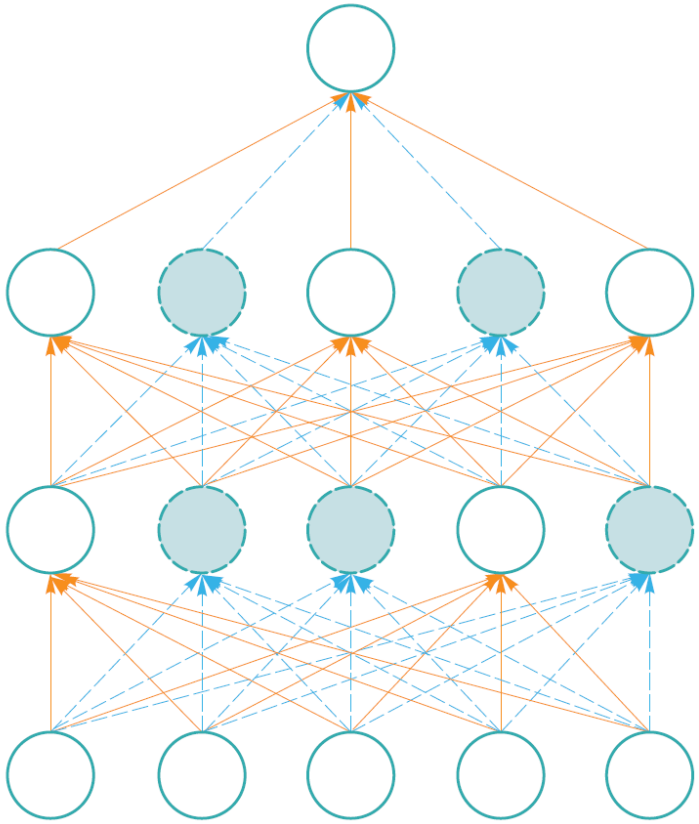


图 5.22　使用 Dropout 的神经网络模型

注意力机制与正则化---正则化

L_1和L_2正则化：对于具有n个训练数据$\{(x_1, y_1), \ldots, (x_n, y_n)\}$的神经网络，其中$(x_i, y_i)(1 \le i \le n)$分别为输入样本及其对应的标签。加入正则化项后，神经网络的损失函数一般可如下表示：

$$\min \frac{1}{n} \sum_{i=1}^{n} \underbrace{Loss\big(y_i, f(\boldsymbol{W}, x_i)\big)}_{\text{损失函数}} + \underbrace{\lambda}_{\text{正则化权重}} \times \underbrace{\Phi(\boldsymbol{W})}_{\text{正则化项}}$$

其中，$f(\boldsymbol{W}, x_i)$表示参数为$\boldsymbol{W}$的神经网络对输入x_i的预测、$\Phi(\boldsymbol{W})$为正则化项（又称惩罚项）、λ为正则项权重。

正则化项$\Phi(\boldsymbol{W})$一般用模型参数$\boldsymbol{W}$的范数形式来表示。假设神经网络模型中参数数目为$\mathbb{N}$，则参数的范数形式主要有：

L_0范数：数学表示为$\|\boldsymbol{W}\|_0 = \sum_{i=1}^{\mathbb{N}} \mathbb{I}[w_i \neq 0]$，其中$\mathbb{I}$是指示函数，若括号内表达式为真，函数值为1，反之为0。所以L_0范数指模型参数$\boldsymbol{W}$中非零元素个数。L_0可实现模型参数的稀疏化。但是，由于L_0范数正则化是一个NP Hard问题，难以求解，因此一般不用L_0范数。

L_1范数：数学表示为$\|W\|_1 = \sum_{i=1}^{\mathbb{N}} |w_i|$，指模型参数$\boldsymbol{W}$中各个元素的绝对值之和。$L_1$范数也被称为“稀疏规则算子”（Lasso regularization）。

L_2范数：数学表示为$\|W\|_2 = \sqrt{\sum_{i=1}^{\mathbb{N}} w_i^2}$，指模型参数$\boldsymbol{W}$中各个元素平方和的开方。

自然语言和计算机视觉应用---词向量生成

1. 文本表征方法

2. 网络结构设计

3. 如何设计学习任务?

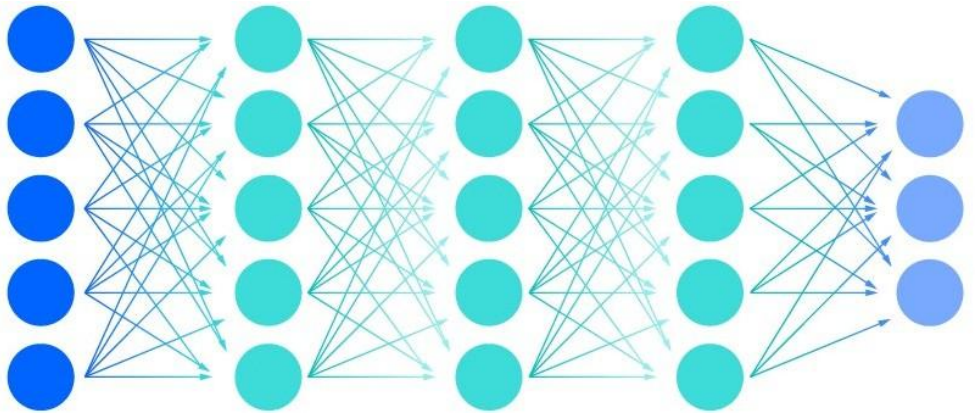
生成任务

AI: “我也是”

“我喜欢这部电影”

向量化

$$\begin{pmatrix} 1 \\ 0.1 \\ 0.5 \\ \vdots \\ -0.2 \end{pmatrix}$$



分类任务

0/1

文本

表征
(Representation)

神经网络

自然语言和计算机视觉应用---词向量生成

词典中有V个不同的单词，如何生成第k个单词的N维词向量?

- 在词袋模型：一个单词按照词典序被表示为一个词典维数大小的向量（称为one-hot vector），向量中该单词所对应位置按照其在文档中出现与否取值为1或0。1表示该单词在文档中出现，0表示没有出现，导致单词间关联丢失
- 词向量（Word2Vec）模型：为了刻画不同单词之间的语义相关性，利用深度学习模型，将每个单词表征为N维实数值的分布式向量

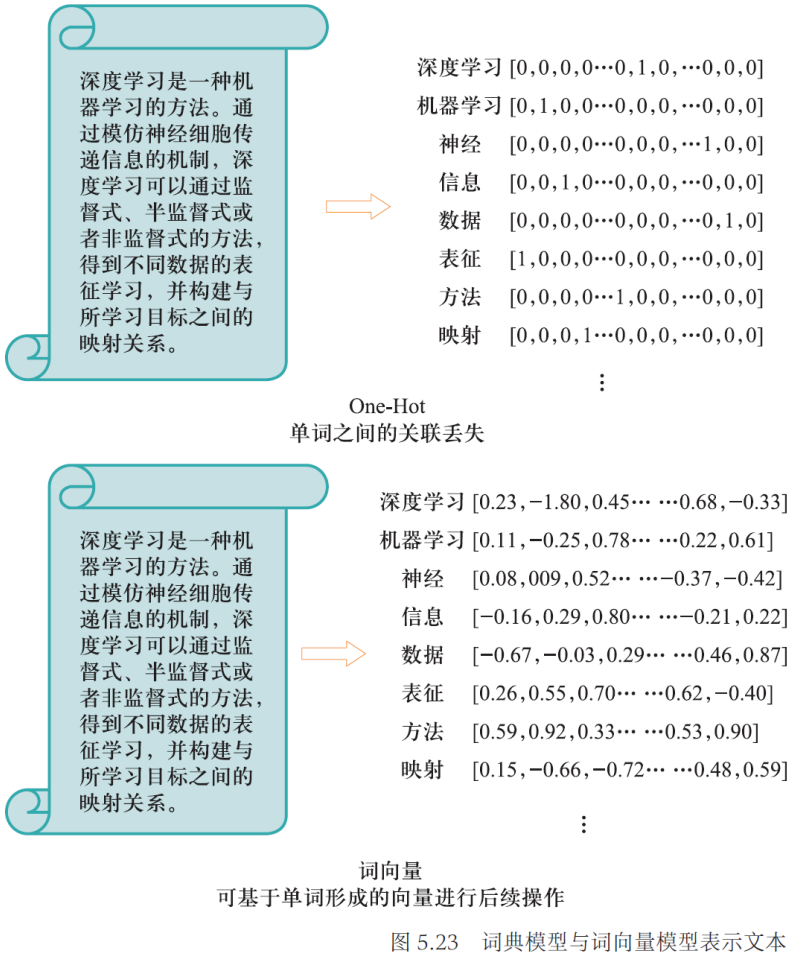


图 5.23　词典模型与词向量模型表示文本

自然语言和计算机视觉应用---词向量生成

词典中有V个不同的单词，如何生成第k个单词的N维词向量?

- 采用词袋模型法将将该单词表示成V维one-hot 向量$\boldsymbol{X}$，向量$\boldsymbol{X}$中第k个位置取值为1、其余位置取值均为0。隐藏层神经元大小为N，每个神经元记为$h_i(1 \le i \le N)$。向量$\boldsymbol{X}$中每个$x_i(1 \le i \le V)$与隐藏层神经元是全连接，连接权重矩阵为$W_{V\times N}$。所得到的隐藏层结果与输出层是全连接，全连接权重矩阵为$W'_{N\times V}$，输出层是V维归一化的概率值，其中y_k对应第k个单词归一化的概率值。

- 这里$W_{V\times N}$和$W'_{N\times V}$为模型参数。一旦训练得到了图5.24的神经网络，就可以将输入层中x_k与隐藏层连接权重为$\{w_{k1}, w_{k2}, \cdots, w_{kN}\}$作为第$k$个单词$N$维词向量(word vector)。

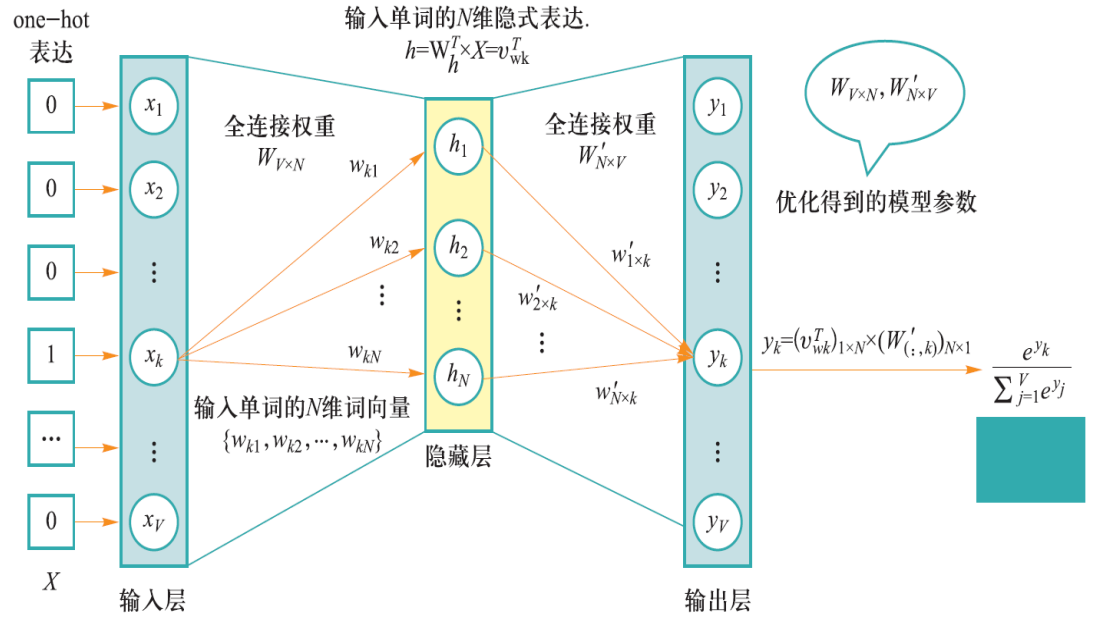
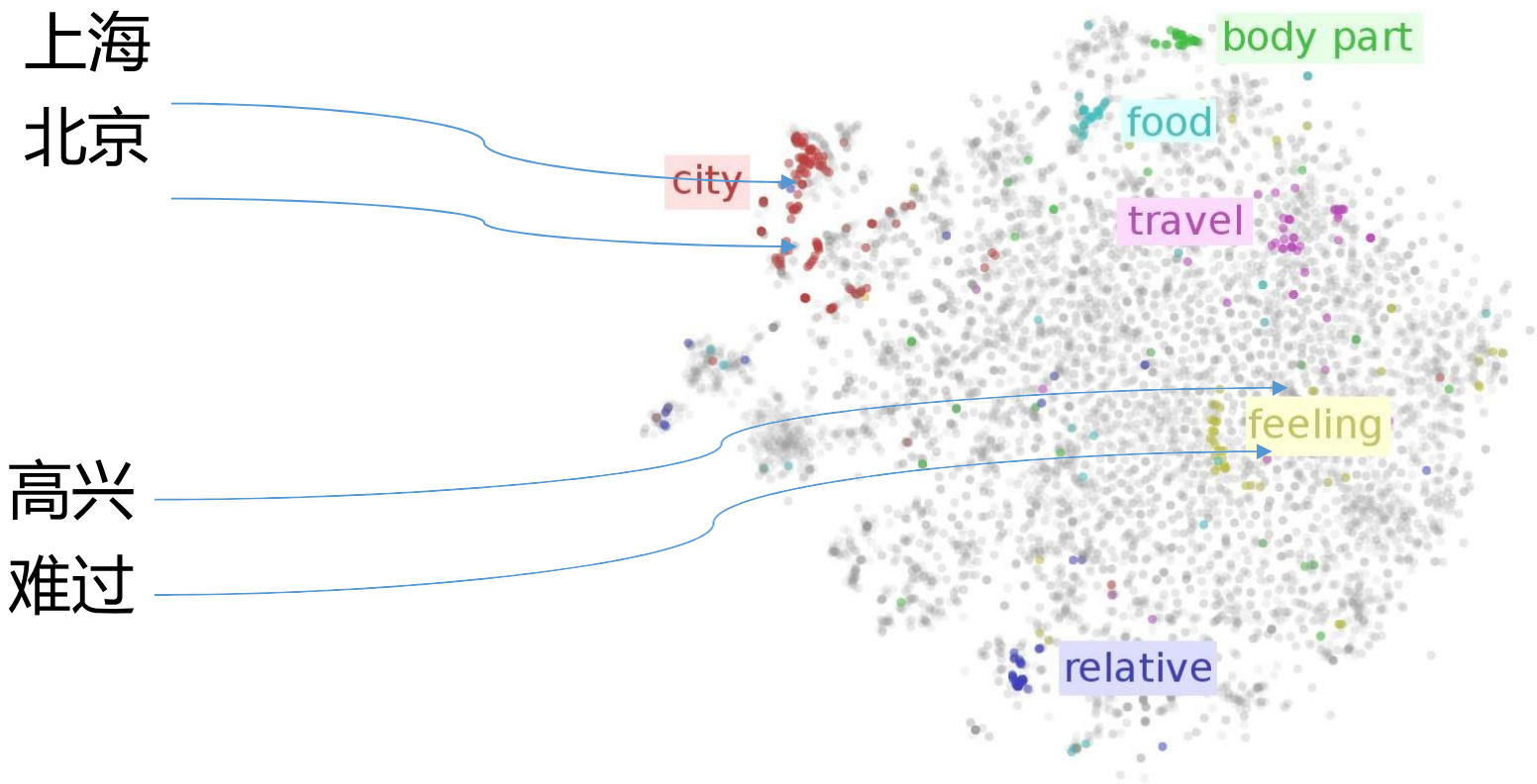


图 5.24　单个单词的词向量生成示意

自然语言和计算机视觉应用---词向量生成

- 词向量空间的距离是语义距离



自然语言和计算机视觉应用---图像分类与目标定位

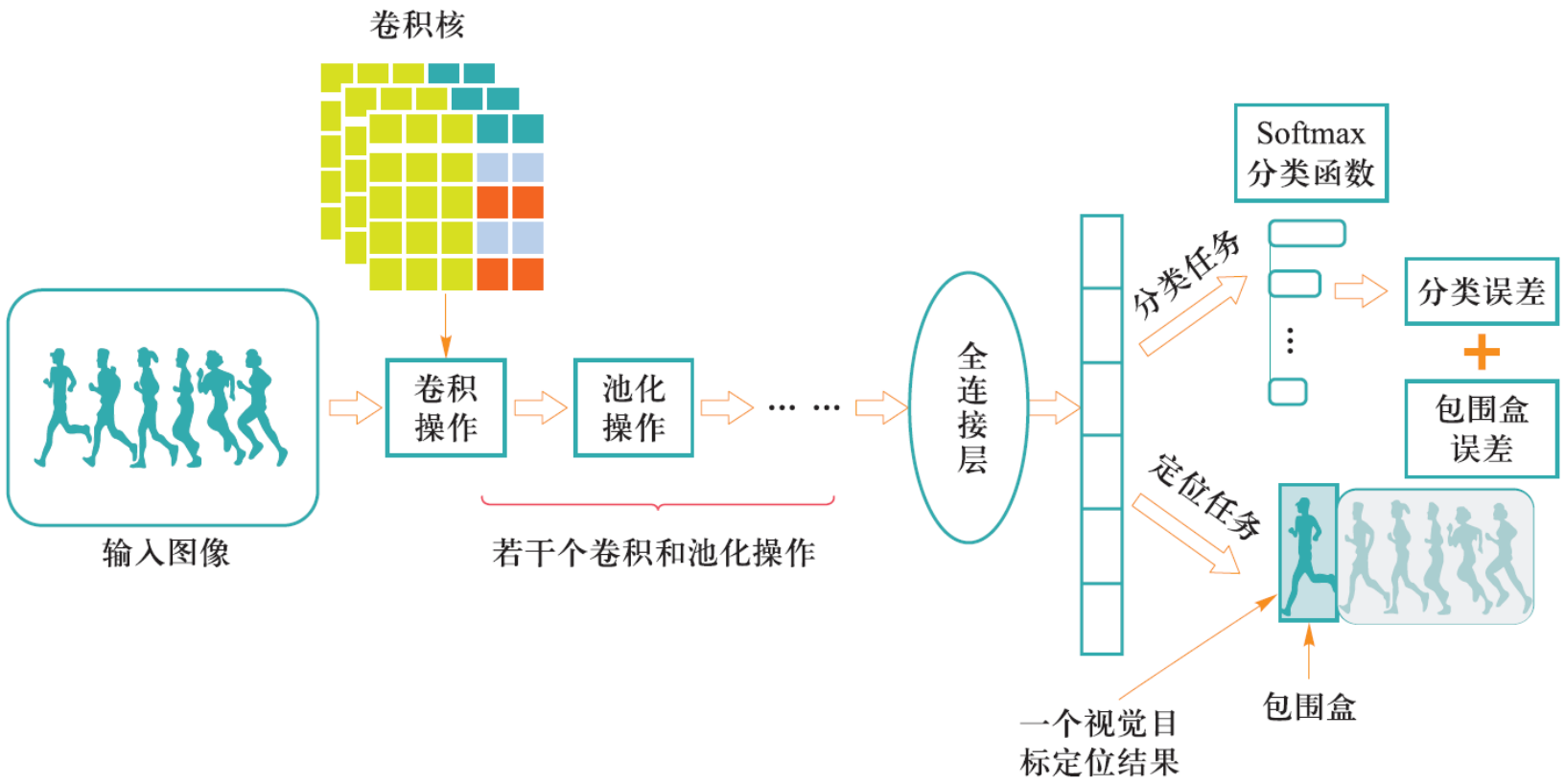


图 5.26　利用卷积神经网络进行图像分类与目标定位示意

a)分类任务。Softmax分类函数将全连接层所得特征向量映射为归一化概率值，用来表示每个概念在输入图像中出现的概率。在训练中，可用交叉熵等损失函数来计算模型学习的误差.
b)目标定位。将全连接层输出向量转换为一个四维向量，四维向量分别代表一个包围盒的左上角横坐标和纵坐标以及该矩形框宽与高取值，并使用均方差损失函数计算模型的误差。最终，将两个任务所得误差相加，作为整个模型的误差